

**KLASIFIKASI SUARA KECELAKAAN LALU LINTAS
MENGUNAKAN METODE *MEL FREQUENCY CEPSTRAL
COEFFICIENTS* (MFCC) DAN *CONVOLUTIONAL NEURAL
NETWORK* (CNN)**

SKRIPSI

Oleh :

REYMANDA PUTRA PERWIRA WARDANY

NPM : 222310072



**PROGRAM STUDI SARJANA TEKNOLOGI INFORMASI
FAKULTAS INFORMATIKA DAN PARIWISATA
INSTITUT BISNIS DAN INFORMATIKA KESATUAN
BOGOR**

2026

**KLASIFIKASI SUARA KECELAKAAN LALU LINTAS
MENGUNAKAN METODE *MEL FREQUENCY CEPSTRAL
COEFFICIENTS* (MFCC) DAN *CONVOLUTIONAL NEURAL
NETWORK* (CNN)**

SKRIPSI

Sebagai salah satu syarat
untuk memperoleh gelar Sarjana Teknologi Informasi
pada Program Studi Sarjana Teknologi Informasi
Institut Bisnis dan Informatika Kesatuan

Oleh :

REYMANDA PUTRA PERWIRA WARDANY

NPM : 222310072



**PROGRAM STUDI SARJANA TEKNOLOGI INFORMASI
FAKULTAS INFORMATIKA DAN PARIWISATA
INSTITUT BISNIS DAN INFORMATIKA KESATUAN
BOGOR**

2026

**KLASIFIKASI SUARA KECELAKAAN LALU LINTAS
MENGUNAKAN METODE *MEL FREQUENCY CEPSTRAL
COEFFICIENTS* (MFCC) DAN *CONVOLUTIONAL NEURAL
NETWORK* (CNN)**

SKRIPSI

Telah diujikan pada:

Hari, Tanggal : Jumat, 31 Juli 2026

Di Institut Bisnis dan Informatika Kesatuan

Mengetahui :

Dekan Fakultas Informatika dan Pariwisata

Ketua Program Studi Sarjana
Teknologi Informasi

H. Bambang Hengky Rainanto,
S.Pi., MM., Ph.D., CSM., CHRP., CIIQA.

Anton Sukanto, S. Kom., M. TI.,
MTCNA., MTCRE

**KLASIFIKASI SUARA KECELAKAAN LALU LINTAS
MENGUNAKAN METODE *MEL FREQUENCY CEPSTRAL
COEFFICIENTS* (MFCC) DAN *CONVOLUTIONAL NEURAL
NETWORK* (CNN)**

SKRIPSI

Telah diseminarkan pada:

Hari, Tanggal : Jumat, 10 Juli 2026

Di Institut Bisnis dan Informatika Kesatuan

Telah disetujui oleh Pembimbing

Isnan Mulia , S.Komp., M.Komp.

Menyetujui :

Penguji I

Penguji II

Dr. Nusa Muktiadji Ir., M.M.

Septian Cahyadi S.Kom., M.Kom.

SURAT PERNYATAAN

Yang bertanda tangan dibawah ini:

Nama : Reymanda Putra Perwira Wardany

NPM : 222310072

Program Studi : Teknologi Informasi – S1

Judul Skripsi : Klasifikasi Suara Kecelakaan Lalu Lintas Menggunakan Metode
Mel Frequency Cepstral Coefficients (MFCC) dan *Convolutional
Neural Network (CNN)*

Menyatakan dengan sebenar-benarnya bahwa segala pernyataan dalam penelitian ini adalah hasil pekerjaan saya sendiri, dengan bimbingan Ketua Pembimbing, kecuali yang jelas ditunjukan rujukannya. Penelitian ini belum pernah diajukan untuk memperoleh gelar pada program sejenis di Perguruan Tinggi lain. Semua sumber data dan informasi yang digunakan telah dinyatakan secara jelas dan dapat diperiksa kebenarannya. Jika ternyata terbukti pernyataan ini tidak benar, sepenuhnya menjadi tanggung jawab saya.

Bogor, Juli 2026

Yang menyatakan

Reymanda Putra
Perwira Wardany

ABSTRAK

Reymanda Putra Perwira. NPM : 222310072 (2026), Klasifikasi Suara Kecelakaan Lalu Lintas Menggunakan Metode *Mel Frequency Cepstral Coefficients* (MFCC) dan *Convolutional Neural Network* (CNN), Di bawah bimbingan ISNAN MULIA

Kecelakaan lalu lintas merupakan salah satu permasalahan yang sering terjadi dan dapat menimbulkan kerugian material maupun korban jiwa. Salah satu pendekatan yang dapat dilakukan untuk mendukung pengembangan sistem pemantauan lalu lintas adalah dengan memanfaatkan klasifikasi suara kecelakaan menggunakan teknologi *deep learning*. Penelitian ini bertujuan untuk membangun model klasifikasi suara kecelakaan lalu lintas menggunakan metode *Mel Frequency Cepstral Coefficients* (MFCC) sebagai teknik ekstraksi fitur dan *Convolutional Neural Network* (CNN) sebagai algoritma klasifikasi. Dataset yang digunakan terdiri dari 600 data *audio* yang terbagi ke dalam enam kelas, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*, dengan masing-masing kelas berjumlah 100 data. Dataset diperoleh dari beberapa sumber, yaitu Freesound, UrbanSound8K, *AudioSet*, dan Pixabay. Sebelum proses pelatihan, seluruh data *audio* melalui tahapan *preprocessing* yang meliputi normalisasi format, penyesuaian durasi menjadi 3 detik, serta ekstraksi fitur menggunakan MFCC dengan 20 koefisien. Model CNN kemudian dilatih menggunakan TensorFlow dan Keras, kemudian dievaluasi menggunakan Confusion Matrix, Accuracy, Precision, *Recall*, dan F1-Score. Hasil penelitian menunjukkan bahwa model CNN yang dibangun mampu mengklasifikasikan enam kategori suara dengan akurasi sebesar 88%. Berdasarkan hasil evaluasi, model menunjukkan performa yang baik dalam mengenali sebagian besar kelas suara, dengan nilai *precision*, *recall*, dan *F1-score* yang tinggi pada sebagian besar kategori. Hasil penelitian ini menunjukkan bahwa kombinasi metode MFCC dan CNN mampu memberikan performa yang baik dalam proses klasifikasi suara kecelakaan lalu lintas sehingga berpotensi menjadi dasar pengembangan sistem klasifikasi suara berbasis kecerdasan buatan pada penelitian selanjutnya.

Kata Kunci: Klasifikasi *Audio*, Kecelakaan Lalu Lintas, MFCC, CNN, Deep Learning.

ABSTRACT

Reymanda Putra Perwira. Student ID: 222310072 (2026 Traffic Accident Sound Classification Using Mel Frequency Cepstral Coefficients (MFCC) and Convolutional Neural Network (CNN), under the supervision of ISNAN MULIA.

Traffic accidents remain a significant issue that can result in property damage, injuries, and loss of life. One approach to supporting the development of intelligent traffic monitoring systems is through traffic accident sound classification using deep learning techniques. This study aims to develop a traffic accident sound classification model by employing Mel Frequency Cepstral Coefficients (MFCC) for feature extraction and Convolutional Neural Network (CNN) as the classification algorithm. The dataset consisted of 600 audio samples distributed across six classes: Crash, Horn, Skid, Normal, Siren, and Engine, with 100 samples for each class. The dataset was collected from Freesound, UrbanSound8K, AudioSet, and Pixabay. Prior to training, all audio data underwent preprocessing, including audio format normalization, duration adjustment to 3 seconds, and MFCC feature extraction using 20 coefficients. The CNN model was developed using TensorFlow and Keras and evaluated using a Confusion Matrix, Accuracy, Precision, Recall, and F1-Score. The experimental results demonstrate that the proposed CNN model achieved an accuracy of 88% in classifying the six sound categories. Evaluation results indicate that the model provides strong performance across most classes, achieving high precision, recall, and F1-score values. These findings suggest that the combination of MFCC and CNN is effective for traffic accident sound classification and has the potential to serve as a foundation for future intelligent audio-based traffic monitoring systems.

Keywords: Audio Classification, Traffic Accident, Mel Frequency Cepstral Coefficients, Convolutional Neural Network, Deep Learning.

KATA PENGANTAR

Puji Syukur kehadiran Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan skripsi ini dengan baik dan tepat waktu. Skripsi ini disusun sebagai salah satu syarat menyelesaikan studi Program Sarjana Teknologi Informasi di Institut Bisnis dan Informatika Kesatuan. Pada kesempatan ini penulis ingin menyampaikan rasa terima kasih yang sebesar-sebesarannya atas bantuan dukungan, dan bimbingannya kepada:

1. Bapak Prof. Dr. Bambang Pamungkas, Ak., MBA, CA, CPA, CPA (Aust), ASEAN CPA, CIMBA, CSFA, CFrA, selaku Rektor Institut Bisnis dan Informatika Kesatuan.
2. Bapak Bambang Hengky Rainanto, S.Pi., M.M., Ph.D. CIIQA. selaku Dekan Fakultas Informatika dan Pariwisata, Institut Bisnis dan Informatika Kesatuan.
3. Bapak Anton Sukamto, S.Kom., M.TI, selaku Kepala Program Studi Teknologi Informasi.
4. Bapak Isnan Mulia S.Komp., M.Kom yang telah memberikan pengajaran dan bimbingan kepada penulis, sehingga penulis dapat mengaplikasikan pengetahuan yang diperoleh dalam penyelesaian skripsi ini.
5. Bapak dan Ibu Dosen Program Studi S1 Teknologi Informasi, Institut Bisnis dan Informatika Kesatuan, yang telah membimbing serta memberikan wawasan berharga kepada Penulis selama proses perkuliahan hingga penyusunan skripsi ini.
6. Rekan-rekan Mahasiswa/i Program Studi Teknologi Informasi Angkatan 2022 dan 2021, yang telah berbagi ilmu serta pengalaman selama perkuliahan dan proses penelitian skripsi ini

Kepada kedua orang tua, Ayah dan Ibu, penulis mengucapkan terima kasih yang sebesar-besarnya atas doa, kasih sayang, dukungan, motivasi, serta pengorbanan yang tidak pernah putus selama penulis menempuh pendidikan hingga menyelesaikan skripsi ini. Segala bentuk dukungan yang telah diberikan menjadi sumber semangat

bagi penulis dalam menghadapi setiap proses dan tantangan selama penyusunan skripsi. Semoga Allah SWT, senantiasa memberikan kesehatan, kebahagiaan, serta keberkahan kepada kedua orang tua atas segala kebaikan yang telah diberikan kepada penulis.

Penulis menyadari bahwa skripsi ini masih memiliki keterbatasan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari berbagai pihak sebagai bahan perbaikan di masa mendatang. Semoga skripsi ini dapat memberikan manfaat bagi pembaca, serta menjadi referensi dan bahan pertimbangan dalam pengembangan penelitian selanjutnya. Dan untuk sebagai penutup, penulis mengucapkan terima kasih.

Bogor, Juli 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	viii
DAFTAR ISI.....	x
DAFTAR GAMBAR	xii
DAFTAR TABEL	xii
DAFTAR LAMPIRAN	xiv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Penelitian	1
1.2 Rumusan Masalah	3
1.3 Tujuan Penelitian	3
1.4 Manfaat Penelitian.....	4
1.5 Ruang Lingkup	4
BAB II TINJAUAN PUSTAKA.....	6
2.1 Landasan Teori	6
2.1.1 Kecelakaan lalu lintas.....	6
2.1.2 Pengolahan sinyal suara	7
2.1.3 Machine learning untuk <i>Audio Classificatiion</i>	7
2.1.4 MFCC (Mel-Frequency Cepstral Coefficient).....	8
2.1.5 <i>Convolutional Neural Network</i> (CNN)	9
2.1.6 TensorFlow dan Keras	9
2.1.7 Google Colaboratory	10
2.1.8 Convusion Matrix.....	11
2.2 Penelitian Terdahulu	13
BAB III METODOLOGI PENELITIAN	17
3.1 Waktu dan Tempat Penelitian	17
3.2 Alat dan Bahan	17
3.3 Prosedur Kerja.....	18
3.3.1 Observasi & Studi Literatur.....	20

3.3.2 Rumusan Masalah	21
3.3.3 Pengumpulan Data.....	22
3.3.4 <i>Preprocessing</i> Data.....	22
3.3.5 Ekstraksi Fitur Menggunakan Mel Frequency Cepstral Coefficient (MFCC)	23
3.3.6 Pembagian Data.....	23
3.3.7 Pembangunan Model CNN.....	24
3.3.8 Pelatihan dan Evaluasi Model	25
3.4 Jadwal Penelitian	26
BAB IV HASIL DAN PEMBAHASAN	28
4.1 Pengumpulan Kebutuhan.....	28
4.2 <i>Preprocessing</i> Data.....	29
4.3 Ekstraksi Fitur	30
4.4 Pembangunan Model CNN.....	32
4.5 Pelatihan dan Model Evaluasi	35
4.5.1 Proses Pelatihan Model	35
4.5.2 Hasil Pengujian Model.....	38
4.5.3 Analisis Confusion Matrix.....	38
4.5.4 Analisis <i>Classification Report</i>	40
BAB V KESIMPULAN DAN SARAN.....	43
5.1 Kesimpulan.....	43
5.2 Saran	44
DAFTAR PUSTAKA.....	45
Lampiran	1

DAFTAR GAMBAR

Gambar 3.1 Kerangka Kerja Model Prototipe	19
Gambar 3.2 Flowchart Sistem Kerja Alat	Error! Bookmark not defined.
Gambar 3.3 Diagram Blok Alur Kerja Sistem	Error! Bookmark not defined.
Gambar 3.4 Desain Fritzing	Error! Bookmark not defined.
Gambar 4.1 flowchart preprocessing data	30
Gambar 4.2 Visualisasi MFCC	31
Gambar 4.3 Diagram Arsitektur CNN	33
Gambar 4.4 Akurasi Pelatihan Model	36
Gambar 4.5 Loss Pelatihan Model	36
Gambar 4.6 Confusion Matrix	39

DAFTAR TABEL

Tabel 2.1 Penelitian Terdahulu.....	13
Tabel 3.1 Alat	17
Tabel 3.2 Bahan.....	18
Tabel 3.3 Jadwal Penelitian.....	26
Tabel 4.1 Jumlah Dataset	28
Tabel 4.2 Parameter Ekstraksi.....	31
Tabel 4.3 Konfigurasi Model CNN.....	33
Tabel 4.5 Hasil Pengujian Model.....	38
Tabel 4.6 Performa Model	41

DAFTAR LAMPIRAN

Lampiran 1 Pelatihan Model CNN	49
Lampiran 2 Training dan Validation Accuracy	49
Lampiran 3 Training dan Validation Loss	50
Lampiran 4 Classification Report	50
Lampiran 5 Convusion Matrix	51
Lampiran 6 Splitting Data	51
Lampiran 7 Ekstrak Dataset	53
Lampiran 8 Bangun Arsitektur CNN	55
Lampiran 9 Training Model CNN	56
Lampiran 10 Evaluasi Model	57
Lampiran 11 Classification Report	57
Lampiran 12 Convusion Matrix	58

BAB I

PENDAHULUAN

1.1 Latar Belakang Penelitian

Transportasi merupakan salah satu sektor yang pertumbuhannya terus meningkat seiring bertambahnya kebutuhan mobilitas masyarakat Indonesia. Jumlah mobil penumpang tercatat sekitar 15,8 juta unit pada 2020, naik menjadi 16,41 juta unit pada 2021, dan mencapai 17,17 juta unit pada 2022 (BPS, 2025). Pertumbuhan volume kendaraan yang tidak diimbangi dengan kesiapan infrastruktur dan kedisiplinan berkendara pada akhirnya turut mendorong tingginya angka kecelakaan lalu lintas. Data Kementerian Perhubungan mencatat 94.617 kasus kecelakaan pada 2022, meningkat 34,6% dibandingkan 70.000 kasus pada 2021 (Kemenhub, 2023). Kondisi serupa juga terjadi pada ruas jalan tol, di mana Korlantas Polri (2024) mencatat 1.464 kecelakaan dengan 688 korban meninggal dunia pada 2022, yang kemudian bertambah menjadi 1.656 kecelakaan dengan 704 korban meninggal dunia pada 2023.

Tingginya angka kecelakaan tersebut erat kaitannya dengan lambatnya penanganan di lapangan. Banyak kejadian baru diketahui petugas setelah ada laporan dari pengguna jalan lain, sementara titik-titik rawan di jalan tol tidak selalu berada dalam jangkauan pengawasan kamera secara langsung. Keterlambatan informasi ini berdampak pada tertundanya pertolongan terhadap korban, padahal kecepatan respons menjadi salah satu faktor penentu keselamatan korban kecelakaan. Kondisi ini mendorong berbagai penelitian mengarah pada pengembangan metode deteksi kejadian kecelakaan yang tidak bergantung pada laporan manual, salah satunya melalui pemanfaatan sinyal *audio* di sekitar lokasi kejadian.

Pendekatan berbasis suara dipandang relevan karena peristiwa kecelakaan umumnya menghasilkan karakteristik akustik khas, seperti benturan keras, decitan ban, maupun bunyi klakson yang berbeda dari suara lalu lintas normal. Islam dan Abdel-Aty (2022) menunjukkan bahwa kombinasi *Convolutional Neural Network* (CNN) dengan fitur Mel-Frequency Cepstral Coefficients (MFCC) mampu membedakan suara

kecelakaan dari kebisingan lalu lintas dengan akurasi yang baik. Temuan serupa disampaikan Singh et al. (2023), yang membuktikan bahwa deteksi kecelakaan jalan berbasis pengolahan sinyal *audio* dan deep learning dapat dijalankan secara real-time dengan hasil yang menjanjikan, sementara Lee dan Kim (2020) memperlihatkan bahwa metode ini bahkan dapat diimplementasikan pada perangkat dengan sumber daya komputasi terbatas. Kumar dan Raj (2020) turut menegaskan bahwa CNN yang dilatih pada fitur *audio* efektif digunakan untuk mendeteksi anomali suara secara cepat, sehingga pendekatan ini berpotensi diadopsi sebagai dasar sistem pemantauan lalu lintas berbasis kecerdasan buatan.

Secara teknis, MFCC dipilih karena mampu merepresentasikan karakteristik spektral sinyal *audio* dengan cara yang mendekati persepsi pendengaran manusia, sehingga banyak digunakan sebagai teknik ekstraksi fitur pada berbagai penelitian klasifikasi suara (Li et al., 2020; Zhang et al., 2021). Fitur yang dihasilkan MFCC kemudian dapat dipelajari polanya oleh CNN, yang dikenal unggul dalam mengenali pola spasial pada data yang direpresentasikan sebagai citra atau matriks fitur, termasuk pada tugas klasifikasi suara lingkungan (Virtanen et al., 2020; Zhao et al., 2024). Kombinasi kedua metode ini pada dasarnya bukan hal baru, namun sebagian besar penelitian terdahulu masih berfokus pada klasifikasi dua kelas, yaitu membedakan suara kecelakaan dengan suara bukan kecelakaan secara umum.

Kondisi lalu lintas di jalan tol pada kenyataannya jauh lebih kompleks. Selain suara tabrakan, sistem juga akan menangkap suara klakson, decitan ban saat pengereman mendadak, mesin kendaraan besar, sirine kendaraan darurat, hingga kondisi lalu lintas yang berjalan normal. Keberagaman sumber suara ini berpotensi menimbulkan kesalahan klasifikasi apabila model hanya dilatih untuk membedakan dua kategori, sehingga dibutuhkan model yang mampu mengenali lebih banyak kelas suara secara spesifik agar hasil klasifikasi lebih merepresentasikan kondisi nyata di lapangan.

Berdasarkan uraian tersebut, penelitian ini bermaksud membangun model klasifikasi suara kecelakaan lalu lintas dengan memanfaatkan MFCC sebagai teknik ekstraksi fitur dan CNN sebagai algoritma klasifikasi pada enam kategori suara, yaitu

Crash, Horn, Skid, Normal, Siren, dan Engine. Dataset yang digunakan merupakan gabungan dari beberapa sumber terbuka, yaitu Freesound, UrbanSound8K, *AudioSet*, dan Pixabay, yang kemudian melalui tahapan *preprocessing* sebelum digunakan untuk pelatihan model. Performa model selanjutnya dievaluasi menggunakan confusion matrix, accuracy, precision, *recall*, dan F1-score, sehingga hasil penelitian ini diharapkan dapat menjadi acuan awal bagi pengembangan sistem klasifikasi suara kecelakaan lalu lintas berbasis kecerdasan buatan pada penelitian selanjutnya.

1.2 Rumusan Masalah

Berdasarkan penjelasan yang tertera di latar belakang tersebut, penulis memfokuskan penelitian berkaitan dengan:

1. Bagaimana membangun model klasifikasi suara kecelakaan lalu lintas menggunakan metode *Mel Frequency Cepstral Coefficients* (MFCC) dan *Convolutional Neural Network* (CNN)?
2. Bagaimana performa model CNN dalam mengklasifikasikan suara *Crash, Horn, Skid, Normal, Siren, dan Engine* berdasarkan nilai accuracy, precision, *recall*, F1-score, dan confusion matrix?
3. Bagaimana kemampuan model dalam membedakan suara kecelakaan lalu lintas dengan suara non-kecelakaan berdasarkan hasil pengujian yang dilakukan?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah penulis memfokuskan tujuan penelitian sebagai berikut:

1. Membangun model klasifikasi suara kecelakaan lalu lintas menggunakan metode MFCC sebagai ekstraksi fitur CNN sebagai algoritma klasifikasi
2. Mengevaluasi performa model CNN berdasarkan nilai *accuracy, precision, recall, F1-score*, dan *confusion matrix*
3. Menganalisis kemampuan model dalam membedakan suara kecelakaan lalu lintas dengan suara non-kecelakaan berdasarkan hasil pengujian.

1.4 Manfaat Penelitian

Adapun manfaat dari penelitian ini yaitu:

Manfaat praktis :

1. Memberikan gambaran mengenai penerapan metode MFCC dan CNN dalam klasifikasi suara kecelakaan lalu lintas.
2. Menjadi referensi bagi mahasiswa atau peneliti yang ingin mengembangkan sistem deteksi kecelakaan berbasis suara.
3. Menjadi dasar pengembangan sistem deteksi kecelakaan secara real-time pada penelitian selanjutnya.

1.5 Ruang Lingkup

Agar penelitian lebih terarah dan sesuai dengan tujuan, ruang lingkup penelitian ini dibatasi pada hal-hal berikut:

1. Penelitian difokuskan pada pembangunan model klasifikasi suara kecelakaan lalu lintas menggunakan metode MFCC dan CNN.
2. Data yang digunakan terdiri atas enam kelas suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*
3. Dataset diperoleh dari beberapa sumber daring, yaitu Freesound, *AudioSet* Google, dan Pixabay, kemudian dilakukan proses seleksi dan *preprocessing* sebelum digunakan dalam pelatihan model.
4. Seluruh *audio* diproses menggunakan sample rate 16.000 Hz, durasi 3 detik, format mono, dan diekstraksi menggunakan 20 koefisien MFCC.
5. Model dibangun menggunakan framework TensorFlow Keras pada platform Google Colaboratory.
6. Evaluasi model dilakukan menggunakan accuracy, loss, precision, *recall*, F1-score, dan confusion matrix.

7. Penelitian ini tidak membahas implementasi model pada perangkat keras maupun sistem deteksi secara real-time.
8. Penelitian ini belum menggunakan K-Fold untuk mengukur kinerja machine learning

IBI KESATUAN BOGOR

BAB II

TINJAUAN PUSTAKA

2.1 Landasan Teori

2.1.1 Kecelakaan lalu lintas

Kecelakaan lalu lintas adalah suatu peristiwa di jalan yang melibatkan kendaraan, pengendara, penumpang, atau pejalan kaki yang mengakibatkan kerugian, baik berupa kerusakan harta benda, luka ringan, luka berat, hingga korban jiwa. Menurut Undang-Undang No 22 Tahun 2009 tentang Lalu lintas dan Angkutan Jalan (LLAJ) kecelakaan lalu lintas adalah peristiwa di jalan yang tidak diduga dan tidak disengaja melibatkan dengan atau tanpa pengguna jalan lain yang mengakibatkan korban manusia dan/atau kerugian harta benda.

Menurut data dari Korlantas Polri, hingga Agustus 2024 telah terjadi sebanyak 79.220 kasus kecelakaan di seluruh Indonesia (Korlantas Polri, 2024). Jumlah ini menunjukkan bahwa tingkat kecelakaan masih tinggi dan memerlukan sistem deteksi serta respon yang lebih cepat.

Selain itu, menurut ketua komite nasional keselamatan transportasi mencatat bahwa 60% kecelakaan moda darat disebabkan oleh faktor manusia, seperti kelelahan dan mengantuk (detiknews, 2024). Hal ini menegaskan pentingnya sistem pendeteksian dini yang dapat memberikan peringatan lebih cepat untuk meminimalkan dampak fatal akibat keterlambatan penanganan.

Dengan data yang begitu banyak mengenai permasalahan kecelakaan, ada pula faktor keterlambatan dalam pelaporan kecelakaan dikarenakan proses pelaporan masih berbasis manual, Sehingga menghasilkan keterlambatan pula dalam penanganan kecelakaan. seperti menurut Korlantas polri informasi kecelakaan umumnya diperoleh dari pengguna jalan atau patroli polisi, bukan dari sistem otomatis, hal ini menyebabkan jeda waktu antara kejadian dan laporan resmi ke pihak berwenang (Korlantas Polri, 2024).

2.1.2 Pengolahan sinyal suara

Sinyal *audio* merupakan bentuk dari representasi digital dari gelombang suara yang dihasilkan oleh getaran partikel udara dan dapat di analisa secara matematis. Analisis sinyal *audio* dapat dilakukan dalam dua domain utama, yaitu domain waktu dan domain frekuensi.

Pada domain waktu, sinyal digambarkan sebagai perubahan amplitudo terhadap waktu yang menunjukkan seberapa kuat suara pada setiap satuan waktu. Analisis ini bermanfaat untuk mendeteksi perubahan energi atau lonjakan amplitudo yang khas pada suara impulsif seperti tabrakan. Pada domain frekuensi, sinyal *audio* dianalisis dengan mengubah representasi sinyal dari domain waktu ke domain frekuensi menggunakan algoritma Fast Fourier Transform (FFT), sehingga komponen frekuensi yang terkandung dalam suara dapat diidentifikasi dan dianalisis secara lebih rinci (Lyons, 2020). Analisis frekuensi memungkinkan sistem mengenali ciri khas suara berdasarkan distribusi frekuensi dan intensitasnya.

Transformasi sinyal dari domain waktu ke domain frekuensi menjadi dasar dalam proses ekstraksi fitur *audio* modern, salah satunya Mel-Frequency Cepstral Coefficient (MFCC), yang dirancang untuk merepresentasikan karakteristik spektral suara sesuai dengan persepsi pendengaran manusia (Virtanen, Plumbley, & Ellis, 2020).

2.1.3 Machine learning untuk *Audio Classification*

Perkembangan *Machine Learning* (ML) telah memungkinkan computer mengenali dan mengklasifikasikan berbagai jenis suara dengan akurasi tinggi. Dalam konteks deteksi kecelakaan lalu lintas berbasis suara, *machine learning* berperan penting untuk membedakan antara suara normal di jalan (seperti, mesin, klakson, atau knalpot) dengan suara impulsive khas tabrakan.

Pendekatan berbasis deep learning terbukti mampu memberikan performa yang lebih baik dalam tugas klasifikasi data kompleks dibandingkan metode pembelajaran mesin konvensional. Deep learning merupakan pengembangan dari machine learning yang memanfaatkan jaringan saraf tiruan berlapis (deep neural networks) untuk secara

otomatis mempelajari representasi fitur tingkat tinggi dari data seperti citra, *audio*, dan teks (Goodfellow, Bengio, & Courville, 2016; Chollet, 2021).

Dalam penelitian ini digunakan arsitektur *Convolutional Neural Network* (CNN), yang dikenal efektif dalam memproses data dua dimensi seperti spectrogram dan fitur MFCC. CNN mampu mengekstraksi pola spasial pada representasi waktu–frekuensi *audio*, sehingga dapat membedakan karakteristik spektral antara suara lalu lintas normal dan suara kecelakaan secara lebih akurat (Virtanen et al., 2020; Müller, 2021).

2.1.4 MFCC (Mel-Frequency Cepstral Coefficient)

Mel-Frequency Cepstral Coefficient (MFCC) merupakan salah satu metode ekstraksi ciri yang paling banyak digunakan dalam pengolahan sinyal *audio* dan pengenalan pola suara. MFCC bertujuan untuk mengubah sinyal suara mentah menjadi representasi numerik yang mencerminkan karakteristik spektral *audio* sesuai dengan persepsi pendengaran manusia (Virtanen, Plumbley, & Ellis, 2020).

Secara konseptual, MFCC dirancang dengan meniru mekanisme sistem pendengaran manusia, di mana persepsi terhadap frekuensi bersifat non-linear. Pendengaran manusia lebih sensitif terhadap perubahan frekuensi rendah dibandingkan frekuensi tinggi, sehingga analisis frekuensi dalam MFCC dilakukan menggunakan skala Mel yang mendekati karakteristik persepsi auditori manusia (Müller, 2021).

MFCC digunakan secara luas karena mampu mengekstraksi fitur suara yang paling representative, khususnya pada aplikasi seperti:

- Pengenalan suara (*speech revognition*).
- Deteksi emosi atau kejadian suara (*sound event detection*).
- Identifikasi suara kecelakaan dalam sistem pengawasan lalu lintas (Islam & Abdel-Aty, 2022).

Kemudian keunggulan dari MFCC mampu menangkap ciri penting dari suara meskipun terdapat kebisingan, ringan secara komputasi, sehingga cocok diterapkan pada perangkat embedded seperti ESP32.

2.1.5 Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN) merupakan salah satu arsitektur *deep learning* yang dirancang untuk melakukan ekstraksi fitur secara otomatis dari data masukan. CNN bekerja dengan memanfaatkan beberapa lapisan (*Layer*) yang saling terhubung sehingga mampu mempelajari pola data secara bertingkat (*hierarchical feature learning*). Dibandingkan metode *machine learning* konvensional, CNN memiliki keunggulan dalam mengenali karakteristik data tanpa memerlukan proses ekstraksi fitur secara manual, sehingga banyak diterapkan pada berbagai bidang seperti klasifikasi citra, pengenalan suara, serta klasifikasi *audio*. (Zhao et al., 2024).

Pada penelitian ini, CNN digunakan sebagai algoritma klasifikasi untuk mengenali enam kategori suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*. Data yang telah melalui proses ekstraksi fitur menggunakan MFCC dijadikan sebagai masukan bagi model CNN untuk mempelajari karakteristik setiap kelas suara. Selama proses pelatihan, CNN melakukan penyesuaian bobot secara iteratif agar mampu menghasilkan prediksi dengan tingkat akurasi yang optimal. Pendekatan ini telah banyak digunakan pada penelitian klasifikasi *audio* karena mampu mempelajari pola lokal yang terdapat pada representasi fitur *audio* secara efektif.

Secara umum, arsitektur CNN terdiri atas beberapa lapisan utama, yaitu *Convolution Layer*, *Activation Function*, *Batch Normalization*, *Pooling Layer*, *Flatten Layer*, *Fully Connected Layer*, dan *Output Layer*. Setiap lapisan memiliki fungsi yang berbeda namun saling berkaitan dalam proses ekstraksi fitur hingga menghasilkan keputusan klasifikasi.

2.1.6 TensorFlow dan Keras

TensorFlow merupakan framework open-source yang dikembangkan oleh Google untuk membangun, melatih, dan mengimplementasikan model machine learning maupun deep learning. TensorFlow menyediakan berbagai pustaka dan antarmuka pemrograman yang memudahkan pengembang dalam melakukan pemrosesan data, membangun arsitektur jaringan saraf, hingga melakukan evaluasi model. Selain itu, TensorFlow mendukung komputasi pada CPU, GPU, maupun TPU

sehingga mampu mempercepat proses pelatihan model, terutama pada dataset yang berukuran besar (TensorFlow, 2023).

Keras merupakan *Application Programming Interface* (API) tingkat tinggi yang terintegrasi dengan TensorFlow dan dirancang untuk menyederhanakan proses pengembangan model *deep learning*. Melalui Keras, proses perancangan arsitektur jaringan saraf dapat dilakukan dengan lebih mudah menggunakan komponen-komponen seperti *Layer*, *loss function*, *optimizer*, dan *evaluation metrics*. Pendekatan ini memungkinkan pengembang membangun model dengan kode yang lebih ringkas, mudah dipahami, serta tetap memiliki performa yang tinggi (TensorFlow, 2023).

Pada penelitian ini, TensorFlow digunakan sebagai *framework* utama dalam proses pembangunan dan pelatihan model CNN, sedangkan Keras dimanfaatkan sebagai antarmuka untuk merancang arsitektur model, melakukan proses pelatihan (*training*), validasi (*validation*), serta evaluasi performa model. Penggunaan TensorFlow dan Keras dipilih karena menyediakan berbagai fungsi yang mendukung implementasi algoritma *deep learning* secara efisien serta kompatibel dengan lingkungan pengembangan berbasis Python (TensorFlow, 2023).

TensorFlow dan Keras juga menyediakan berbagai fungsi untuk menyimpan model yang telah dilatih, melakukan prediksi terhadap data baru, serta mendukung integrasi dengan berbagai platform pengembangan lainnya. Kemampuan tersebut menjadikan TensorFlow dan Keras sebagai salah satu *framework* yang banyak digunakan dalam penelitian maupun pengembangan aplikasi berbasis kecerdasan buatan, khususnya pada bidang klasifikasi citra dan *audio* (TensorFlow, 2023).

2.1.7 Google Colaboratory

Google Colaboratory atau Google Colab merupakan lingkungan pengembangan berbasis *cloud* yang dikembangkan oleh Google untuk menjalankan kode Python secara langsung melalui peramban (*web browser*) tanpa memerlukan proses instalasi perangkat lunak pada komputer pengguna. Platform ini dibangun di atas Jupyter Notebook sehingga memungkinkan pengguna untuk menulis, menjalankan, mendokumentasikan, serta membagikan kode program dalam satu

dokumen interaktif. Selain itu, Google Colab menyediakan akses terhadap sumber daya komputasi seperti CPU, GPU, dan TPU yang dapat dimanfaatkan untuk mempercepat proses pelatihan model *machine learning* maupun *deep learning*.

Google Colab banyak dimanfaatkan dalam penelitian di bidang kecerdasan buatan karena menyediakan lingkungan pengembangan yang mudah digunakan, mendukung berbagai pustaka pemrograman Python, serta terintegrasi dengan layanan Google Drive. Integrasi tersebut memudahkan pengguna dalam menyimpan dataset, notebook, maupun hasil pelatihan model secara daring sehingga proses penelitian dapat dilakukan secara lebih fleksibel dan kolaboratif. Selain itu, notebook yang dibuat pada Google Colab dapat diakses dari berbagai perangkat selama terhubung dengan jaringan internet.

Pada penelitian ini, Google Colab digunakan sebagai lingkungan pengembangan untuk seluruh proses pembangunan model klasifikasi suara kecelakaan lalu lintas. Seluruh tahapan penelitian, mulai dari proses *preprocessing* data *audio*, ekstraksi fitur menggunakan MFCC, pelatihan model CNN, hingga evaluasi performa model dilakukan menggunakan Google Colab. Penggunaan platform ini dipilih karena mampu menyediakan sumber daya komputasi yang memadai serta mendukung integrasi dengan TensorFlow dan Keras sehingga proses pengembangan model dapat dilakukan secara lebih efisien.

Google Colab juga mendukung penggunaan berbagai pustaka *machine learning* dan *deep learning* yang telah tersedia secara bawaan atau dapat dipasang dengan mudah menggunakan Python Package Installer (*pip*). Dukungan tersebut memungkinkan proses implementasi model menjadi lebih sederhana serta mempercepat proses eksperimen dan pengujian selama penelitian berlangsung.

2.1.8 Confusion Matrix

Confusion Matrix merupakan salah satu metode evaluasi yang digunakan untuk mengukur kinerja model klasifikasi dengan membandingkan hasil prediksi model terhadap data aktual. Melalui Confusion Matrix, dapat diketahui jumlah data yang berhasil maupun gagal diklasifikasikan pada setiap kelas sehingga memberikan

informasi yang lebih rinci mengenai performa model dibandingkan hanya menggunakan nilai akurasi. Oleh karena itu, metode ini banyak digunakan dalam evaluasi model *machine learning* maupun *deep learning*, termasuk pada klasifikasi *audio* dan *citra* (Provost & Fawcett, 2023; Google Developers, 2024).

Pada permasalahan klasifikasi multikelas (*multiclass classification*), Confusion Matrix disajikan dalam bentuk matriks yang terdiri atas baris sebagai kelas aktual dan kolom sebagai kelas prediksi. Setiap sel pada matriks menunjukkan jumlah data yang diprediksi ke dalam suatu kelas tertentu. Semakin banyak nilai yang berada pada diagonal utama matriks, maka semakin baik kemampuan model dalam melakukan klasifikasi terhadap setiap kelas (Google Developers, 2024).

Selain Confusion Matrix, performa model juga dievaluasi menggunakan beberapa metrik pengukuran, yaitu Accuracy, Precision, *Recall*, dan F1-Score. Keempat metrik tersebut digunakan untuk memberikan gambaran yang lebih komprehensif mengenai kemampuan model dalam melakukan proses klasifikasi sehingga hasil evaluasi tidak hanya bergantung pada satu indikator saja (Sokolova & Lapalme, 2021).

2.1.8.1 Accuracy

Accuracy merupakan metrik evaluasi yang digunakan untuk mengukur tingkat ketepatan model dalam melakukan klasifikasi terhadap seluruh data uji. Nilai accuracy diperoleh dari perbandingan antara jumlah prediksi yang benar dengan jumlah keseluruhan data yang diuji. Semakin tinggi nilai accuracy, maka semakin baik kemampuan model dalam mengklasifikasikan data (Google Developers, 2024).

2.1.8.2 Precision

Precision digunakan untuk mengukur tingkat ketepatan model dalam memberikan prediksi positif. Nilai precision menunjukkan seberapa banyak data yang diprediksi sebagai suatu kelas benar-benar berasal dari kelas tersebut. Metrik ini penting ketika kesalahan prediksi positif perlu diminimalkan (Google Developers, 2024).

2.1.8.3 Recall

Recall merupakan metrik yang digunakan untuk mengukur kemampuan model dalam menemukan seluruh data yang benar-benar berasal dari suatu kelas. Nilai *recall* yang tinggi menunjukkan bahwa model mampu mengenali sebagian besar data pada kelas tersebut sehingga jumlah data yang tidak terdeteksi menjadi lebih kecil (Google Developers, 2024).

2.1.8.4 F1-Score

F1-Score merupakan nilai yang diperoleh dari rata-rata harmonik antara *precision* dan *recall*. Metrik ini digunakan ketika diperlukan keseimbangan antara kemampuan model dalam memberikan prediksi yang tepat dan kemampuan model dalam menemukan seluruh data pada suatu kelas. Nilai F1-Score berada pada rentang 0 hingga 1, di mana nilai yang semakin mendekati 1 menunjukkan performa model yang semakin baik (Sokolova & Lapalme, 2021).

2.2 Penelitian Terdahulu

Penelitian terdahulu yang berkaitan dengan topik ini telah dilakukan oleh beberapa peneliti, masing-masing menyotot berbagai aspek penting dan relevan yang memberikan landasan teoritis serta hasil-hasil empiris yang berharga, yang menjadi acuan dalam mengembangkan penelitian ini lebih lanjut

Tabel 2.1 Penelitian Terdahulu

Penelitian Ke-1	
Judul	<i>Deep Convolutional Neural Network For Roadmy Incident Surveillance Using Audio Data</i>
Tahun	2022
Penulis	Zubayer Islam & Mohamed Abdel-Aty

Ringkasan	Menggunakan CNN dengan fitur MFCC dan log Mel-filterbank untuk mengenali suara kecelakaan, skid, klakson, dan sirine. Akurasi mencapai 94%.
-----------	---

Kesimpulan	Penelitian ini berbasis komputer dengan dataset besar dan lingkungan terkendali, belum diterapkan pada sistem real-time embedded seperti ESP32. Penelitian mengisi celah ini dengan implementasi lapangan real-time.
------------	--

Penelitian Ke-2

Judul	<i>Environmental Sound Classification for Traffic Accident Detection Using Convolutional Neural Networks</i>
-------	--

Tahun	2021
-------	------

Penulis	Zhang, Liu, & Wang
---------	--------------------

Ringkasan	Penelitian ini menggunakan <i>Convolutional Neural Network</i> (CNN) untuk mengklasifikasikan suara lingkungan jalan, termasuk suara tabrakan, klakson, dan kebisingan lalu lintas. Data <i>audio</i> diubah menjadi representasi spectrogram dan MFCC sebelum diproses oleh CNN. Hasil eksperimen menunjukkan bahwa CNN mampu mengenali suara kecelakaan dengan akurasi di atas 93%.
-----------	---

Kesimpulan	Pendekatan CNN terbukti unggul dalam mengenali pola spektral suara kecelakaan. Namun, sistem masih berjalan pada komputer dengan sumber daya besar dan belum dioptimalkan untuk implementasi real-time pada perangkat embedded. Penelitian mengadaptasi metode ini agar dapat berjalan pada ESP32 dengan sumber daya terbatas.
------------	--

Penelitian Ke-3

Judul	<i>Real-Time Road Accident Detection Based on Audio Signal Processing and Deep Learning</i>
-------	---

Tahun	2023
-------	------

Penulis	Singh, Kumar, & Verma
Ringkasan	Penelitian ini mengusulkan sistem deteksi kecelakaan lalu lintas secara real-time menggunakan analisis sinyal <i>audio</i> dan deep learning. Fitur MFCC digunakan sebagai masukan ke model CNN untuk membedakan suara kecelakaan dengan suara normal jalan raya. Sistem diuji pada berbagai kondisi lalu lintas dan menunjukkan performa stabil dengan tingkat akurasi tinggi. membutuhkan perangkat komputer dengan sumber daya besar.
Kesimpulan	Penelitian ini menegaskan bahwa kombinasi MFCC dan CNN efektif untuk deteksi kecelakaan berbasis <i>audio</i> secara real-time. Namun, implementasi masih berfokus pada sistem berbasis komputer. Penelitian memperluas penelitian ini dengan integrasi langsung ke sistem IoT berbasis ESP32 dan aktivasi sirine otomatis sebagai respons keselamatan.
Penelitian Ke-4	
Judul	<i>Detection of Traffic Accident Sound Using Machine Learning on Raspberry Pi</i>
Tahun	2020
Penulis	Lee & Kim
Ringkasan	Menggunakan Raspberry Pi dengan SVM untuk mengenali suara tabrakan berbasis MFCC. Akurasi 91% pada pengujian indoor.
Kesimpulan	Sistem masih menggunakan komputer mini (Raspberry), belum optimal untuk IoT skala luas. Penelitian lebih ringan dan murah karena berbasis ESP32.
Penelitian Ke-5	
Judul	<i>Audio-Based Traffic Accident Detection Using Deep Learning and Spectrogram Analysis</i>
Tahun	2022
Penulis	Islam & Abdel-Aty

Ringkasan Penelitian ini mengembangkan sistem deteksi kecelakaan lalu lintas berbasis suara menggunakan pendekatan deep learning. Sinyal *audio* diubah ke dalam bentuk log-Mel spectrogram dan MFCC sebelum diproses menggunakan *Convolutional Neural Network* (CNN). Sistem diuji pada berbagai jenis suara lalu lintas seperti tabrakan, pengereman mendadak, klakson, dan sirine. Hasil pengujian menunjukkan akurasi deteksi mencapai lebih dari 94%.

Kesimpulan Hasil penelitian menunjukkan bahwa CNN sangat efektif dalam mengenali pola suara kecelakaan dari representasi spektral *audio*. Namun, sistem masih berbasis komputer dengan dataset besar dan belum diimplementasikan pada perangkat embedded berdaya rendah. Penelitian melengkapi kekurangan ini dengan mengimplementasikan sistem deteksi suara kecelakaan secara real-time pada perangkat ESP32 yang lebih sederhana dan ekonomis.

BAB III

METODOLOGI PENELITIAN

3.1 Waktu dan Tempat Penelitian

Dalam perancangan proposal skripsi ini, waktu dan tempat yang sesuai telah dijadwalkan. Penelitian ini direncanakan akan dimulai bulan Agustus 2025 sampai bulan Mei 2026 dan tempat pelaksanaannya adalah di rumah, kampus, dan gerbang tol. Hal ini bertujuan untuk meningkatkan efisiensi dalam pengambilan data dan waktu penelitian.

3.2 Alat dan Bahan

Alat yang digunakan dalam penelitian ini meliputi perangkat keras dan perangkat lunak sebagai pendukung proses pengolahan data, pelatihan model, serta evaluasi hasil. Adapun spesifikasi perangkat keras yang digunakan adalah sebagai berikut:

Tabel 3.1 *Hardware*

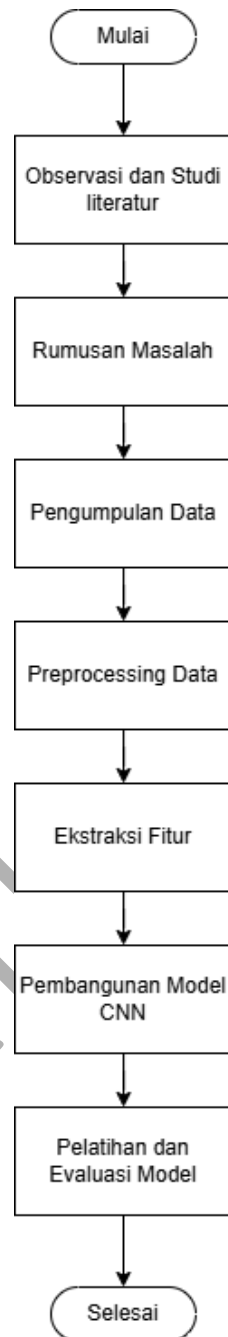
Nama <i>Hardware</i>	Keterangan
Platform Komputasi	Google Collab
Processor	AMD Ryzen 5 5625U with Radeon Graphics
Sistem Operasi	Windows 11
RAM	16 GB
Penyimpanan	474 GB

Tabel 3.2 Software

Nama Software	Fungsi
Python	Bahasa Pemrograman yang digunakan dalam penelitian
Tensorflow	Framework untuk membangun dan melatih model CNN
Keras	API pada tensorflow yang digunakan untuk membangun arsitektur CNN
Librosa	Melakukan <i>Preprocessing audio</i> dan ekstraksi fitur MFCC
Numpy	Mengolah data numerik dan array
Scikit-learn	Pembagian dataset serta evaluasi model
Matplotlib	Menampilkan grafik hasil pelatihan dan confusion matrix
Google Drive	Menyimpan dataset dan model hasil penelitian

3.3 Prosedur Kerja

Penelitian ini menggunakan metode penelitian kuantitatif dengan pendekatan eksperimen (experimental research). Pendekatan kuantitatif digunakan karena hasil penelitian dianalisis berdasarkan data numerik berupa nilai accuracy, precision, *recall*, F1-score, dan confusion matrix. Sementara itu, metode eksperimen digunakan untuk membangun, melatih, dan menguji model klasifikasi suara kecelakaan berbasis *Mel Frequency Cepstral Coefficients* (MFCC) dan *Convolutional Neural Network* (CNN) guna mengetahui performa model dalam mengklasifikasikan enam kelas suara.



Gambar 3.1 Prosedur kerja

Berdasarkan Gambar 3.1, prosedur kerja penelitian dilakukan secara bertahap mulai dari observasi dan studi literatur hingga evaluasi model. Tahap pertama adalah observasi dan studi literatur, yaitu mengidentifikasi permasalahan penelitian serta

mengumpulkan referensi yang berkaitan dengan klasifikasi *audio*, metode ekstraksi fitur MFCC, dan algoritma CNN. Tahap berikutnya adalah perumusan masalah untuk menentukan fokus penelitian serta tujuan yang ingin dicapai.

Selanjutnya dilakukan pengumpulan dataset *audio* yang terdiri dari enam kelas suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*. Dataset yang telah dikumpulkan kemudian melalui tahap *preprocessing* untuk menyeragamkan format *audio*, meliputi konversi ke format WAV, resampling menjadi 16 kHz, konversi menjadi mono, dan penyesuaian durasi *audio* menjadi 3 detik.

Tahap berikutnya adalah ekstraksi fitur menggunakan *Mel Frequency Cepstral Coefficients* (MFCC) untuk memperoleh representasi numerik dari karakteristik *audio*. Hasil ekstraksi fitur kemudian digunakan sebagai masukan pada proses pembangunan model *Convolutional Neural Network* (CNN). Model yang telah dibangun selanjutnya dilatih menggunakan data latih dan diuji menggunakan data uji untuk mengetahui kemampuan model dalam mengklasifikasikan enam kelas suara.

Tahap terakhir adalah evaluasi model, yang dilakukan menggunakan metrik *accuracy*, *precision*, *recall*, *F1-score*, dan *confusion matrix*. Hasil evaluasi kemudian dianalisis untuk mengetahui performa model serta menjadi dasar dalam penarikan kesimpulan penelitian.

3.3.1 Observasi & Studi Literatur

Pada tahap ini dilakukan observasi terhadap permasalahan yang menjadi dasar penelitian, yaitu masih tingginya angka kecelakaan lalu lintas serta perlunya sistem yang mampu mengidentifikasi suara kecelakaan secara otomatis. Observasi dilakukan dengan mengidentifikasi karakteristik berbagai jenis suara yang sering dijumpai pada kondisi lalu lintas, seperti suara benturan kendaraan (*Crash*), klakson (*Horn*), gesekan ban (*skid*), *Sirene* kendaraan darurat (*Siren*), suara mesin kendaraan (*Engine*), serta suara lalu lintas normal (*normal*). Hasil observasi digunakan sebagai dasar dalam menentukan kelas-kelas suara yang akan digunakan pada penelitian.

Selain observasi, dilakukan pula studi literatur dengan mengkaji berbagai referensi seperti jurnal ilmiah, buku, artikel, dan publikasi penelitian yang berkaitan

dengan pengolahan sinyal *audio*, ekstraksi fitur menggunakan *Mel Frequency Cepstral Coefficients* (MFCC), serta metode klasifikasi menggunakan *Convolutional Neural Network* (CNN). Studi literatur bertujuan untuk memperoleh landasan teori, memahami metode yang telah digunakan pada penelitian terdahulu, serta menentukan pendekatan yang paling sesuai untuk diterapkan dalam penelitian ini.

Melalui tahap observasi dan studi literatur, diperoleh pemahaman mengenai proses klasifikasi *audio*, teknik ekstraksi fitur, serta algoritma klasifikasi yang menjadi dasar dalam penyusunan metodologi penelitian dan pengembangan model yang digunakan.

3.3.2 Rumusan Masalah

Berdasarkan hasil observasi dan studi literatur, diperoleh informasi bahwa suara yang dihasilkan saat terjadi kecelakaan lalu lintas memiliki karakteristik yang berbeda dengan suara kendaraan pada kondisi normal maupun suara lain yang sering dijumpai di lingkungan jalan. Perbedaan karakteristik tersebut memungkinkan dilakukannya proses klasifikasi menggunakan teknik pengolahan sinyal *audio* dan metode pembelajaran mesin.

Pada tahap ini dilakukan perumusan masalah sebagai dasar pelaksanaan penelitian. Permasalahan yang dirumuskan adalah bagaimana membangun model klasifikasi suara berbasis *Mel Frequency Cepstral Coefficients* (MFCC) dan *Convolutional Neural Network* (CNN) yang mampu mengklasifikasikan enam jenis suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*. Selain itu, penelitian ini juga bertujuan untuk mengetahui tingkat performa model yang dibangun melalui proses evaluasi menggunakan metrik *accuracy*, *precision*, *recall*, *F1-score*, dan *confusion matrix*.

Hasil dari tahap perumusan masalah menjadi acuan dalam menentukan kebutuhan dataset, tahapan pengolahan data, metode ekstraksi fitur, arsitektur model CNN, serta metode evaluasi yang digunakan selama proses penelitian.

3.3.3 Pengumpulan Data

Pada tahap ini dilakukan pengumpulan dataset *audio* yang akan digunakan sebagai data penelitian. Dataset diperoleh dari beberapa sumber yang menyediakan *audio* secara bebas untuk keperluan penelitian, yaitu Freesound, UrbanSound8K, Google *AudioSet*, Pixabay, serta beberapa *audio* yang diperoleh dari YouTube dan kemudian dipotong menjadi beberapa klip sesuai dengan kebutuhan penelitian. Penggunaan beberapa sumber dataset bertujuan untuk memperoleh variasi karakteristik suara sehingga model mampu mempelajari pola suara yang lebih beragam.

Dataset yang digunakan terdiri atas enam kelas suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*. Masing-masing kelas memiliki 100 data *audio*, sehingga total dataset yang digunakan dalam penelitian ini berjumlah 600 data *audio*. Seluruh data dikumpulkan dengan memperhatikan kesesuaian karakteristik suara pada setiap kelas agar dapat merepresentasikan kondisi yang sebenarnya.

Sebelum digunakan pada tahap berikutnya, seluruh dataset diperiksa untuk memastikan kualitas *audio*, menghindari data yang duplikat, serta memastikan setiap file dapat diproses dengan baik. Selanjutnya, seluruh data dikelompokkan ke dalam folder berdasarkan masing-masing kelas untuk mempermudah proses *preprocessing*, pembagian dataset, dan pelatihan model.

3.3.4 Preprocessing Data

Setelah proses pengumpulan dataset selesai dilakukan, tahap berikutnya adalah *preprocessing* data *audio*. Tahap ini bertujuan untuk menyeragamkan karakteristik seluruh data *audio* agar memiliki format yang konsisten sebelum dilakukan proses ekstraksi fitur. *Preprocessing* merupakan tahapan penting karena kualitas data masukan akan memengaruhi performa model klasifikasi yang dibangun.

Pada penelitian ini, *preprocessing* dilakukan menggunakan bahasa pemrograman Python pada platform Google Colaboratory dengan bantuan pustaka Librosa. Setiap file *audio* diproses melalui beberapa tahapan, yaitu mengubah format *audio* menjadi WAV, melakukan resampling menjadi 16.000 Hz, mengubah *audio* menjadi mono, serta menyeragamkan durasi *audio* menjadi 3 detik. Apabila durasi *audio* lebih dari 3 detik maka dilakukan proses trimming, sedangkan *audio* yang

memiliki durasi kurang dari 3 detik akan ditambahkan padding hingga mencapai durasi yang telah ditentukan.

Tahapan *preprocessing* tersebut dilakukan untuk memastikan seluruh data *audio* memiliki ukuran dan karakteristik yang seragam sehingga proses ekstraksi fitur menggunakan MFCC dapat berjalan dengan optimal. Hasil dari *preprocessing* berupa dataset *audio* yang siap digunakan pada tahap ekstraksi fitur.

3.3.5 Ekstraksi Fitur Menggunakan Mel Frequency Cepstral Coefficient (MFCC)

Setelah seluruh data *audio* berhasil melalui tahap *preprocessing*, langkah selanjutnya adalah melakukan ekstraksi fitur menggunakan metode *Mel Frequency Cepstral Coefficients* (MFCC). Tahap ini bertujuan untuk mengubah sinyal *audio* menjadi representasi numerik yang dapat diproses oleh model klasifikasi *Convolutional Neural Network* (CNN).

MFCC dipilih karena mampu merepresentasikan karakteristik spektral suara sesuai dengan cara kerja sistem pendengaran manusia. Selain itu, metode ini banyak digunakan dalam penelitian klasifikasi *audio* karena efektif dalam menangkap informasi penting dari sinyal suara serta mampu membedakan karakteristik antar kelas *audio*.

Proses ekstraksi fitur dilakukan menggunakan pustaka Librosa pada bahasa pemrograman Python. Setiap file *audio* hasil *preprocessing* diproses untuk menghasilkan matriks MFCC yang selanjutnya digunakan sebagai data masukan bagi model CNN.

3.3.6 Pembagian Data

Sebelum data *audio* digunakan dalam proses pelatihan model, dataset yang telah melalui tahap *preprocessing* perlu dibagi menjadi beberapa subset agar performa model dapat diukur secara objektif. Pembagian data (*data splitting*) bertujuan untuk memisahkan data yang digunakan untuk melatih model, menyetel *hyperparameter*, dan menguji kemampuan generalisasi model terhadap data yang belum pernah dilihat sebelumnya.

Pada penelitian ini, dataset dibagi menjadi tiga bagian, yaitu data latih (*training set*), data validasi (*validation set*), dan data uji (*testing set*), dengan proporsi masing-masing sebesar 80%, 10%, dan 10%. Pembagian ini dilakukan menggunakan fungsi `train_test_split` dari pustaka *scikit-learn*, dengan tahapan sebagai berikut: seluruh data pada setiap kelas terlebih dahulu dibagi menjadi 80% data latih dan 20% data sisa. Selanjutnya, 20% data sisa tersebut dibagi kembali secara merata menjadi 10% data validasi dan 10% data uji.

Proses pembagian data dilakukan secara terpisah untuk setiap kelas (*Crash, Horn, skid, normal, Siren, Engine*), bukan pada keseluruhan dataset secara langsung. Pendekatan ini bertujuan untuk menjaga proporsi jumlah data pada setiap kelas tetap seimbang di ketiga subset, sehingga mencegah terjadinya ketimpangan distribusi kelas (*class imbalance*) yang dapat memengaruhi objektivitas evaluasi model.

Selain itu, parameter `random_state=42` digunakan pada proses pembagian data untuk memastikan hasil pembagian bersifat konsisten dan dapat direproduksi (*reproducible*) apabila proses yang sama dijalankan kembali. Parameter `shuffle=True` juga diterapkan agar data diacak terlebih dahulu sebelum dibagi, sehingga urutan data pada dataset asli tidak memengaruhi hasil pembagian.

3.3.7 Pembangunan Model CNN

Pada tahap ini dilakukan pembangunan model klasifikasi menggunakan *Convolutional Neural Network* (CNN). Model CNN dibangun berdasarkan hasil ekstraksi fitur *Mel Frequency Cepstral Coefficients* (MFCC) yang telah diperoleh pada tahap sebelumnya. Fitur MFCC digunakan sebagai data masukan karena mampu merepresentasikan karakteristik suara dalam bentuk numerik sehingga dapat diproses oleh jaringan saraf tiruan.

Pembangunan model dilakukan menggunakan bahasa pemrograman Python dengan framework TensorFlow dan API Keras pada platform Google Colaboratory. Model CNN dirancang untuk melakukan klasifikasi terhadap enam kelas suara, yaitu *Crash, Horn, Skid, Normal, Siren, dan Engine*.

Arsitektur CNN yang digunakan terdiri dari beberapa lapisan, yaitu *Input Layer*, *Convolution Layer*, *Batch Normalization*, *Max Pooling Layer*, *Flatten Layer*, *Dense Layer*, *Dropout Layer*, serta *Output Layer* dengan fungsi aktivasi *Softmax*. Setiap lapisan memiliki fungsi masing-masing dalam mengekstraksi pola, mengurangi dimensi data, serta menghasilkan prediksi kelas berdasarkan karakteristik suara yang dipelajari oleh model.

Model yang telah dibangun kemudian dipersiapkan untuk memasuki tahap pelatihan dan evaluasi guna mengetahui tingkat performanya dalam mengklasifikasikan data *audio*.

3.3.8 Pelatihan dan Evaluasi Model

Setelah model *Convolutional Neural Network* (CNN) berhasil dibangun, tahap selanjutnya adalah melakukan proses pelatihan (*training*) dan evaluasi model. Pada penelitian ini, proses pelatihan dan evaluasi dilakukan menggunakan metode *Stratified 5-Fold Cross Validation*. Implementasi metode tersebut dilakukan menggunakan pustaka *scikit-learn* pada bahasa pemrograman Python melalui kelas *StratifiedKFold*, sehingga proses pembagian data pada setiap *fold* dilakukan secara otomatis dengan tetap mempertahankan proporsi jumlah data pada setiap kelas. Metode ini membagi seluruh dataset menjadi lima subset (*fold*) yang memiliki distribusi kelas seimbang. Pada setiap iterasi, satu *fold* digunakan sebagai data pengujian (*testing*), sedangkan empat *fold* lainnya digunakan sebagai data pelatihan (*training*). Proses tersebut dilakukan sebanyak lima kali sehingga setiap *fold* memperoleh kesempatan menjadi data pengujian sebanyak satu kali. Pendekatan ini dipilih agar proses evaluasi model lebih objektif serta mampu menggambarkan kemampuan generalisasi model terhadap data yang belum pernah dipelajari sebelumnya.

Proses pelatihan dilakukan menggunakan dataset hasil ekstraksi fitur *Mel Frequency Cepstral Coefficients* (MFCC) dengan parameter pelatihan yang telah ditentukan pada tahap sebelumnya. Pada setiap iterasi *fold*, model dibangun kembali (*reinitialize*) kemudian dilatih menggunakan data pelatihan yang sesuai dengan pembagian pada iterasi tersebut. Selama proses pelatihan, model memanfaatkan

beberapa *callback*, yaitu Early Stopping, Model Checkpoint, dan ReduceLROnPlateau, untuk memperoleh model terbaik berdasarkan performa selama proses pelatihan serta membantu mengurangi risiko *overfitting*.

Setelah proses pelatihan pada setiap *fold* selesai dilakukan, model dievaluasi menggunakan data pengujian pada *fold* tersebut. Evaluasi dilakukan untuk mengetahui kemampuan model dalam mengklasifikasikan enam kelas suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*. Performa model diukur menggunakan beberapa metrik evaluasi, yaitu accuracy, precision, *recall*, F1-score, dan confusion matrix. Nilai akurasi yang diperoleh pada setiap *fold* kemudian dihitung rata-ratanya sebagai nilai performa akhir model. Hasil evaluasi tersebut selanjutnya dianalisis untuk mengetahui tingkat keberhasilan model serta mengidentifikasi kelas-kelas yang masih mengalami kesalahan klasifikasi.

3.4 Jadwal Penelitian

Jadwal penelitian ini dilakukan selama 9 bulan, dari Agustus 2025 hingga Mei 2026. Tabel berikut menunjukkan jadwal kegiatan penelitian berdasarkan metode SDLC, yang mencakup proses pengumpulan kebutuhan, pembuatan desain, membangun prototipe, evaluasi & perbaikan, dan implementasi.

Tabel 3.3 Jadwal Penelitian

No	Nama Kegiatan	2025					2026						
		Agt	Sep	Okt	Nov	Des	Jan	Feb	Mar	Apr	Mei	juni	juli
1	Observasi dan Studi Literatur												
2	Rumusan Masalah												
3	Data Collection												
4	<i>Preprocessing</i> Data												
5	Ekstraksi fitur												

No	Nama Kegiatan	2025					2026						
		Agt	Sep	Okt	Nov	Des	Jan	Feb	Mar	Apr	Mei	Juni	Juli
6	Pembangunan Model CNN												
7	Pelatihan dan Evaluasi Model												

BAB IV

HASIL DAN PEMBAHASAN

4.1 Pengumpulan Kebutuhan

Tahap Data Collection merupakan proses pengumpulan dataset *audio* yang digunakan sebagai data utama dalam penelitian. Dataset diperoleh dari beberapa sumber yang menyediakan *audio* secara bebas untuk keperluan penelitian, yaitu Freesound, UrbanSound8K, Google *AudioSet*, Pixabay, serta beberapa *audio* yang diperoleh dari YouTube yang kemudian dipotong menjadi beberapa klip sesuai dengan kebutuhan penelitian. Penggunaan beberapa sumber dataset bertujuan untuk memperoleh variasi karakteristik suara sehingga model dapat mempelajari pola *audio* yang lebih beragam.

Dataset yang digunakan terdiri atas enam kelas suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*. Masing-masing kelas terdiri dari 100 data *audio*, sehingga total dataset yang digunakan dalam penelitian ini berjumlah 600 data *audio*. Seluruh data yang telah dikumpulkan kemudian dikelompokkan berdasarkan kelas masing-masing untuk memudahkan proses pengolahan data pada tahapan berikutnya.

Sebelum digunakan dalam proses *preprocessing*, setiap data *audio* diperiksa untuk memastikan kualitas suara, menghindari data yang duplikat, serta memastikan file dapat diproses dengan baik. Selain itu, dilakukan penyesuaian penamaan dan pengelompokan file agar struktur dataset lebih terorganisir dan mempermudah proses pemanggilan data pada saat implementasi program.

Tabel 4.1 Komposisi Dataset

No	Kelas	Jumlat Data
1	<i>Crash</i>	100
2	<i>Horn</i>	100
3	<i>Skid</i>	100
4	<i>Normal</i>	100
5	<i>Siren</i>	100

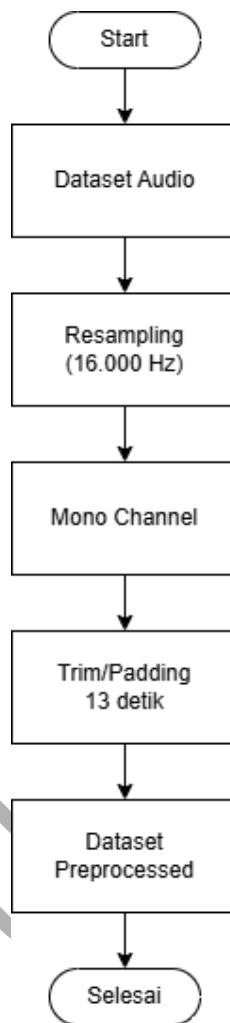
6	<i>Engine</i>	100
Total		600

4.2 Preprocessing Data

Setelah proses pengumpulan dataset selesai dilakukan, tahap berikutnya adalah *preprocessing* terhadap seluruh data *audio*. Tahap ini bertujuan untuk menyeragamkan karakteristik setiap file *audio* agar memiliki format yang konsisten sebelum dilakukan proses ekstraksi fitur menggunakan *Mel Frequency Cepstral Coefficients* (MFCC). Proses *preprocessing* diperlukan karena dataset yang diperoleh berasal dari beberapa sumber sehingga memiliki format file, sample rate, jumlah kanal (channel), dan durasi yang berbeda-beda.

Pada penelitian ini, *preprocessing* dilakukan menggunakan bahasa pemrograman Python pada platform Google Colaboratory dengan memanfaatkan pustaka Librosa. Seluruh data *audio* diproses secara otomatis melalui beberapa tahapan, yaitu konversi format *audio* menjadi WAV, perubahan sample rate menjadi 16.000 Hz, konversi *audio* menjadi mono, serta penyesuaian durasi *audio* menjadi 3 detik. *Audio* yang memiliki durasi lebih dari tiga detik dipotong (trimming), sedangkan *audio* yang berdurasi kurang dari tiga detik ditambahkan padding hingga mencapai durasi yang telah ditentukan.

Tahapan *preprocessing* dilakukan secara konsisten terhadap seluruh 600 data *audio* sehingga setiap file memiliki karakteristik yang sama sebelum memasuki proses ekstraksi fitur. Keceragaman data ini penting untuk menghasilkan fitur MFCC yang konsisten dan membantu meningkatkan stabilitas proses pelatihan model CNN.



Gambar 4.1 flowchart *preprocessing* data

Berdasarkan Gambar 4.3, proses *preprocessing* dilakukan secara bertahap mulai dari konversi format *audio* hingga penyesuaian durasi. Seluruh tahapan tersebut bertujuan untuk menghasilkan dataset dengan karakteristik yang seragam sehingga setiap data dapat diproses secara konsisten pada tahap ekstraksi fitur menggunakan MFCC.

4.3 Ekstraksi Fitur

Setelah seluruh data *audio* selesai melalui tahap *preprocessing*, proses selanjutnya adalah melakukan ekstraksi fitur menggunakan metode *Mel Frequency*

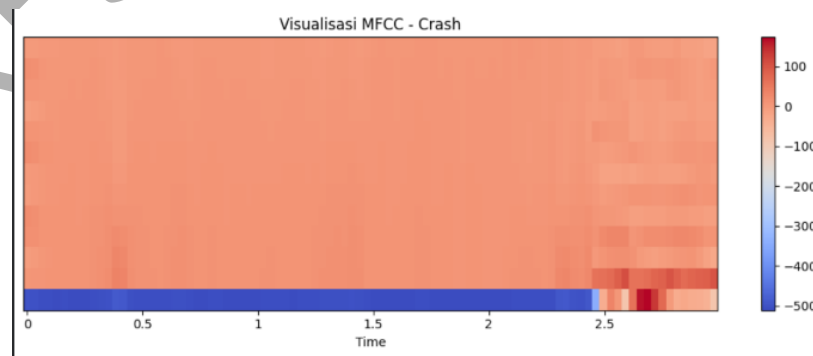
Cepstral Coefficients (MFCC). Tahap ini bertujuan untuk mengubah sinyal *audio* menjadi representasi numerik yang dapat diproses oleh model *Convolutional Neural Network* (CNN). Dibandingkan menggunakan sinyal *audio* secara langsung, fitur MFCC mampu merepresentasikan karakteristik spektral suara sehingga memudahkan model dalam mempelajari pola dari setiap kelas *audio*.

Proses ekstraksi fitur dilakukan menggunakan pustaka Librosa pada bahasa pemrograman Python. Seluruh data *audio* hasil *preprocessing* diekstraksi menggunakan parameter yang sama agar menghasilkan dimensi fitur yang seragam. Parameter ekstraksi yang digunakan dalam penelitian ini ditunjukkan pada Tabel 4.3.

Tabel 4.2 Parameter Ekstraksi

Parameter	Nilai
Sample Rate	16.000 hz
Durasi <i>Audio</i>	3 Detik
N_MFCC	20
N_FFT	2048
Hop_Length	512

Hasil ekstraksi fitur berupa matriks MFCC yang merepresentasikan karakteristik setiap sinyal *audio* dalam domain waktu dan frekuensi. Matriks tersebut kemudian digunakan sebagai data masukan pada model CNN untuk proses klasifikasi.



Gambar 4.2 Visualisasi MFCC

Berdasarkan Gambar 4.2, hasil ekstraksi MFCC memperlihatkan pola distribusi energi yang berbeda pada setiap bagian sinyal *audio*. Pola tersebut menjadi representasi karakteristik suara yang akan dipelajari oleh model CNN selama proses pelatihan. Dengan menggunakan hasil ekstraksi MFCC, informasi penting dari sinyal *audio* dapat dipertahankan, sementara ukuran data menjadi lebih ringkas sehingga proses pelatihan model dapat dilakukan secara lebih efisien.

4.4 Pembangunan Model CNN

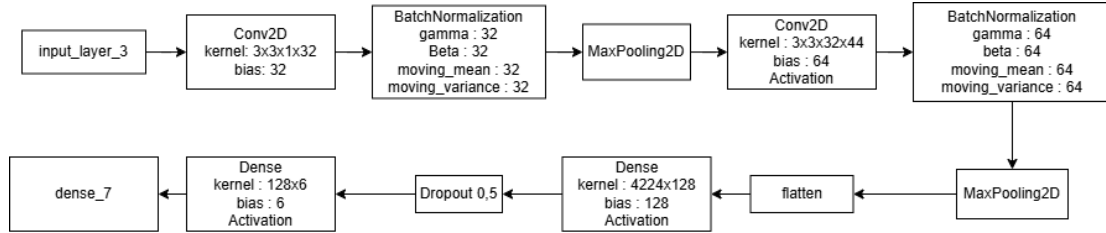
Setelah proses ekstraksi fitur selesai dilakukan, tahap berikutnya adalah membangun model klasifikasi menggunakan *Convolutional Neural Network* (CNN). Model ini dirancang untuk mengenali pola dari hasil ekstraksi fitur MFCC sehingga mampu mengklasifikasikan suara ke dalam enam kelas, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*.

Pembangunan model dilakukan menggunakan framework TensorFlow dengan API Keras pada platform Google Colaboratory. Data hasil ekstraksi MFCC yang telah melalui proses pembagian dataset kemudian digunakan sebagai masukan (input) model. Sebelum proses pelatihan dilakukan, data terlebih dahulu disesuaikan dengan bentuk masukan yang dibutuhkan oleh arsitektur CNN sehingga setiap sampel memiliki dimensi yang seragam.

Arsitektur CNN pada penelitian ini terdiri atas beberapa lapisan, yaitu *Input Layer*, *Convolution Layer*, *Batch Normalization*, *Max Pooling Layer*, *Flatten Layer*, *Dense Layer*, *Dropout Layer*, dan *Output Layer*. Lapisan konvolusi berfungsi untuk mengekstraksi pola dari fitur MFCC, sedangkan lapisan pooling digunakan untuk mengurangi dimensi data tanpa menghilangkan informasi penting. Selanjutnya, hasil ekstraksi fitur diteruskan ke lapisan Dense untuk melakukan proses klasifikasi. Pada lapisan keluaran digunakan fungsi aktivasi Softmax untuk menghasilkan probabilitas prediksi dari enam kelas suara yang digunakan dalam penelitian.

Model kemudian dikompilasi menggunakan optimizer Adam dan fungsi Categorical Crossentropy karena penelitian ini merupakan klasifikasi multikelas.

Konfigurasi tersebut dipilih karena mampu memberikan proses pelatihan yang stabil dan banyak digunakan pada penelitian klasifikasi berbasis CNN.



Gambar 4.3 Diagram Arsitektur CNN

Berdasarkan Gambar 4.3, model klasifikasi dibangun menggunakan arsitektur *Convolutional Neural Network* (CNN) yang terdiri atas beberapa lapisan, yaitu *Input Layer*, dua *Convolutional Layer*, dua *Batch Normalization Layer*, dua *MaxPooling2D Layer*, *Flatten Layer*, *Dense Layer*, *Dropout Layer*, dan *Output Layer*. Lapisan konvolusi berfungsi mengekstraksi pola dari fitur MFCC, sedangkan *Batch Normalization* digunakan untuk menjaga kestabilan proses pelatihan. Selanjutnya, *MaxPooling2D* digunakan untuk mengurangi dimensi fitur tanpa menghilangkan informasi penting. Hasil ekstraksi fitur kemudian diratakan melalui *Flatten Layer* sebelum diteruskan ke *Dense Layer* untuk proses klasifikasi. Pada lapisan keluaran digunakan fungsi aktivasi Softmax untuk menghasilkan probabilitas prediksi terhadap enam kelas suara.

Tabel 4.3 Konfigurasi Model CNN

Parameter	Nilai
Input Shape	(94,20,1)
Convolutional Layer	2
Filter Conv1	32
Filter Conv2	32
Kernel Size	3 x 3
MaxPooling	2 x 3
Dense Layer	128 Neuron

Dropout	0,5
Output Layer	6 Kelas
Fungsi Aktivasi Output	Softmax
Optimizer	Adam
Lost Function	Categorical Crossentropy
Epoch	30
Batch Size	16

Berdasarkan Tabel 4.3, model *Convolutional Neural Network* (CNN) dibangun menggunakan data masukan (*input*) berukuran $94 \times 20 \times 1$, yang merupakan hasil ekstraksi fitur MFCC dari setiap sampel *audio*. Model terdiri atas dua lapisan konvolusi dengan jumlah filter masing-masing 32 dan 64, serta menggunakan ukuran kernel 3×3 untuk mengekstraksi karakteristik dari data masukan. Setelah proses konvolusi, dilakukan proses *max pooling* berukuran 2×2 untuk mengurangi dimensi fitur sekaligus mempertahankan informasi yang penting.

Selanjutnya, hasil ekstraksi fitur diratakan (*flatten*) dan diteruskan ke *Dense Layer* yang terdiri atas 128 neuron. Untuk mengurangi risiko *overfitting*, diterapkan Dropout sebesar 0,5 sebelum proses klasifikasi dilakukan pada lapisan keluaran. Model menggunakan fungsi aktivasi Softmax pada *output Layer* karena penelitian ini melakukan klasifikasi terhadap enam kelas suara.

Pada proses pelatihan, model dikompilasi menggunakan optimizer Adam dengan fungsi *loss* Categorical Crossentropy, *Batch Size* 16, serta jumlah maksimum 30 epoch. Selain itu, diterapkan beberapa mekanisme *callback*, yaitu *EarlyStopping*, *ModelCheckpoint*, dan *ReduceLROnPlateau* untuk meningkatkan efisiensi proses pelatihan, mengurangi risiko *overfitting*, serta menyimpan model dengan performa terbaik selama proses pelatihan. Kombinasi arsitektur dan parameter tersebut dipilih agar model mampu mempelajari karakteristik suara secara optimal sehingga menghasilkan performa klasifikasi yang baik.

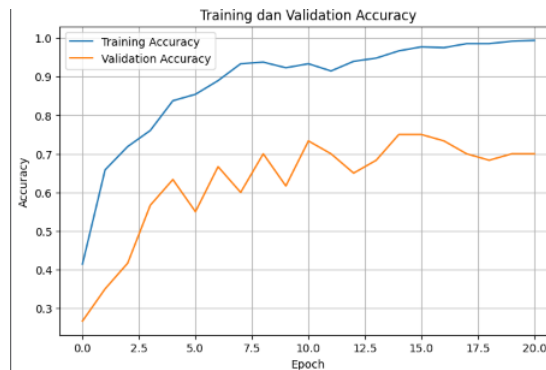
4.5 Pelatihan dan Model Evaluasi

4.5.1 Proses Pelatihan Model

Setelah arsitektur *Convolutional Neural Network* (CNN) berhasil dibangun, tahap selanjutnya adalah melakukan proses pelatihan (*training*) menggunakan data latih yang telah dipersiapkan pada tahap sebelumnya. Sebelum proses pelatihan dimulai, dataset dibagi menggunakan metode hold-out *validation* dengan proporsi 80% data latih, 10% data validasi, dan 10% data uji. Pembagian tersebut bertujuan agar model dapat mempelajari karakteristik data melalui data latih, memantau performa model menggunakan data validasi, serta mengevaluasi kemampuan model terhadap data yang belum pernah digunakan selama proses pelatihan melalui data uji.

Proses pelatihan dilakukan menggunakan 480 data latih, 60 data validasi, dan 60 data uji. Model dilatih menggunakan optimizer Adam, fungsi *loss* Categorical Crossentropy, *Batch Size* 16, dan jumlah maksimum 30 epoch. Selama proses pelatihan diterapkan beberapa mekanisme *callback*, yaitu *EarlyStopping*, *ModelCheckpoint*, dan *ReduceLROnPlateau*. Mekanisme *EarlyStopping* dan *ReduceLROnPlateau* digunakan untuk memantau nilai *validation loss* (*val_loss*) selama proses pelatihan, sedangkan *ModelCheckpoint* digunakan untuk menyimpan model dengan nilai *validation accuracy* (*val_accuracy*) terbaik. Penggunaan *callback* tersebut bertujuan untuk meningkatkan efisiensi proses pelatihan, mengurangi risiko *overfitting*, serta memastikan model terbaik digunakan pada tahap pengujian.

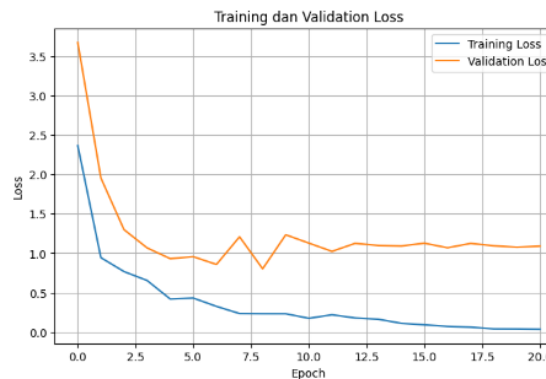
Meskipun jumlah maksimum pelatihan ditetapkan sebanyak 30 epoch, proses pelatihan berhenti secara otomatis pada epoch ke-21 karena mekanisme *EarlyStopping* mendeteksi bahwa nilai *validation loss* tidak lagi mengalami perbaikan selama 12 epoch berturut-turut (*patience* = 12). Berdasarkan hasil pelatihan, model dengan performa terbaik diperoleh pada epoch ke-9, sehingga sistem secara otomatis mengembalikan bobot (*weights*) terbaik melalui mekanisme *restore best weights*. Dengan demikian, model yang digunakan pada tahap evaluasi merupakan model dengan performa validasi terbaik, bukan model pada epoch terakhir. Proses pelatihan model berlangsung selama 43,02 detik



Gambar 4.4 Akurasi Pelatihan Model

Berdasarkan Gambar 4.4, nilai training accuracy mengalami peningkatan secara bertahap sejak awal proses pelatihan. Pada epoch pertama, akurasi pelatihan berada pada kisaran 42%, kemudian meningkat hingga mendekati 99% pada akhir pelatihan. Hal tersebut menunjukkan bahwa model mampu mempelajari pola dari data pelatihan dengan baik.

Sementara itu, *validation accuracy* juga mengalami peningkatan pada beberapa epoch awal, dari sekitar 27% hingga mencapai nilai terbaik sekitar 75%. Setelah itu, nilai *validation accuracy* cenderung berfluktuasi pada kisaran 69%–75% hingga proses pelatihan dihentikan. Fluktuasi tersebut menunjukkan bahwa peningkatan kemampuan model pada data validasi mulai melambat meskipun model masih terus meningkatkan performanya pada data pelatihan.



Gambar 4.5 Loss Pelatihan Model

Berdasarkan Gambar 4.5, nilai *training loss* mengalami penurunan yang cukup signifikan selama proses pelatihan. Pada awal pelatihan, nilai *training loss* berada pada kisaran 2,35, kemudian menurun secara bertahap hingga mendekati 0,05 pada epoch terakhir. Penurunan tersebut menunjukkan bahwa model semakin mampu meminimalkan kesalahan prediksi terhadap data pelatihan seiring bertambahnya jumlah epoch.

Sementara itu, nilai *validation loss* juga mengalami penurunan yang cukup tajam pada beberapa epoch awal, dari sekitar 3,70 hingga mencapai nilai terendah sekitar 0,82 pada epoch ke-9. Setelah mencapai nilai tersebut, *validation loss* mulai mengalami fluktuasi pada kisaran 1,00–1,20 hingga proses pelatihan dihentikan. Kondisi ini menunjukkan bahwa setelah epoch ke-9, peningkatan kemampuan model terhadap data validasi mulai terbatas meskipun model masih terus meningkatkan performanya pada data pelatihan.

Perbedaan pola antara kurva *training loss* dan *validation loss* menunjukkan adanya kecenderungan overfitting ringan. Hal ini terlihat dari nilai *training loss* yang terus menurun secara konsisten, sedangkan *validation loss* tidak lagi mengalami penurunan yang signifikan dan cenderung berfluktuasi. Kondisi tersebut mengindikasikan bahwa model mulai mempelajari karakteristik data pelatihan secara lebih spesifik sehingga kemampuan generalisasinya terhadap data validasi tidak mengalami peningkatan yang sebanding.

Untuk mengurangi dampak kondisi tersebut, penelitian ini menerapkan mekanisme *Early Stopping* dengan parameter *patience* sebanyak 12 epoch. Proses pelatihan dihentikan secara otomatis pada epoch ke-21 karena nilai *validation loss* tidak lagi mengalami perbaikan selama 12 epoch berturut-turut. Selain itu, mekanisme *restore best weights* mengembalikan bobot model terbaik yang diperoleh pada epoch ke-9, sehingga model yang digunakan pada tahap pengujian merupakan model dengan nilai *validation loss* terbaik selama proses pelatihan.

4.5.2 Hasil Pengujian Model

Pengujian ini bertujuan untuk mengetahui kemampuan model dalam mengklasifikasikan data yang belum pernah digunakan selama proses pelatihan sehingga dapat menggambarkan kemampuan generalisasi model terhadap data baru.

Hasil pengujian menunjukkan bahwa model memperoleh test accuracy sebesar 81,33% dengan *test loss* sebesar 0,5906. Selain itu, waktu yang dibutuhkan model untuk melakukan proses pengujian terhadap seluruh data uji adalah 0,2437 detik. Hasil tersebut menunjukkan bahwa model mampu melakukan proses klasifikasi dengan waktu yang relatif singkat serta memiliki tingkat akurasi yang cukup baik dalam membedakan enam kelas suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*.

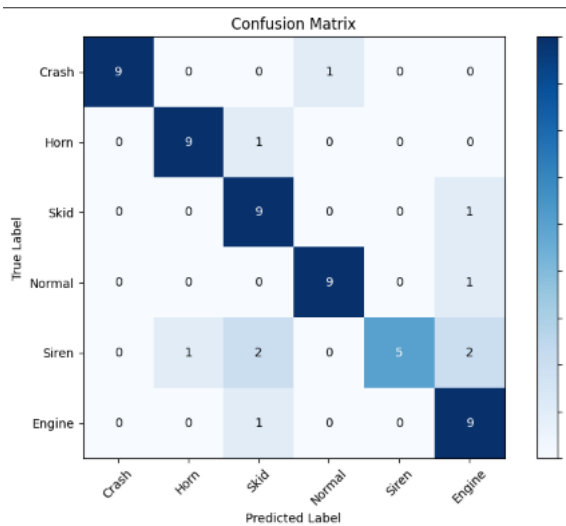
Tabel 4.4 Hasil Pengujian Model

Parameter	Hasil
Test Accuracy	81,33%
Test Loss	0,5906
Waktu Testing	0,2437 detik

Berdasarkan Tabel 4.5, nilai test accuracy sebesar 81,33% menunjukkan bahwa sebagian besar data uji berhasil diklasifikasikan dengan benar oleh model. Sementara itu, nilai *test loss* sebesar 0,5906 mengindikasikan bahwa model telah mempelajari pola dari data dengan cukup baik sehingga kesalahan prediksi yang dihasilkan relatif rendah. Waktu pengujian yang hanya membutuhkan 0,2437 detik menunjukkan bahwa model memiliki proses inferensi yang cepat.

4.5.3 Analisis Confusion Matrix

Selain menggunakan nilai *accuracy*, *precision*, *recall*, dan *F1-score*, evaluasi performa model juga dilakukan menggunakan confusion matrix. Confusion matrix digunakan untuk mengetahui jumlah prediksi yang benar maupun yang salah pada setiap kelas, sehingga dapat memberikan gambaran mengenai kemampuan model dalam membedakan masing-masing kategori suara.



Gambar 4.6 *Confusion Matrix*

Berdasarkan Gambar 4.6, model menunjukkan kemampuan klasifikasi yang cukup baik terhadap enam kelas suara yang digunakan dalam penelitian. Sebagian besar data uji berhasil diklasifikasikan dengan benar yang ditunjukkan oleh nilai diagonal pada confusion matrix. Namun demikian, masih terdapat beberapa kesalahan klasifikasi pada beberapa kelas yang memiliki karakteristik suara yang saling menyerupai.

Pada kelas *Crash*, model berhasil mengklasifikasikan 9 dari 10 data dengan benar, sedangkan 1 data salah diprediksi sebagai kelas Normal. Hal ini menunjukkan bahwa karakteristik suara tabrakan telah dapat dikenali dengan baik oleh model meskipun masih terdapat sedikit kemiripan dengan suara pada kondisi normal.

Pada kelas *Horn*, model juga berhasil mengklasifikasikan 9 dari 10 data secara benar, sedangkan 1 data salah diprediksi sebagai kelas Skid. Kesalahan tersebut mengindikasikan bahwa pada beberapa kondisi terdapat kemiripan karakteristik frekuensi antara suara klakson dan suara gesekan ban.

Kelas Skid memperoleh hasil klasifikasi yang cukup baik dengan 9 dari 10 data berhasil dikenali secara benar. Sebanyak 1 data lainnya salah diklasifikasikan sebagai

kelas *Engine*, yang menunjukkan adanya kemiripan karakteristik suara antara gesekan ban dan suara mesin pada beberapa sampel.

Pada kelas Normal, model berhasil mengenali 9 dari 10 data secara tepat, sedangkan 1 data salah diprediksi sebagai kelas *Engine*. Kesalahan ini menunjukkan bahwa beberapa suara kendaraan pada kondisi normal memiliki karakteristik yang menyerupai suara mesin sehingga masih menimbulkan kesalahan klasifikasi.

Kelas *Siren* merupakan kelas yang memiliki tingkat kesalahan klasifikasi paling tinggi. Dari 10 data uji, hanya 5 data yang berhasil diklasifikasikan dengan benar, sedangkan 1 data salah diprediksi sebagai kelas *Horn*, 2 data sebagai kelas Skid, dan 2 data sebagai kelas *Engine*. Hasil tersebut menunjukkan bahwa model masih mengalami kesulitan dalam membedakan karakteristik suara *Sirene* dengan beberapa jenis suara kendaraan lainnya.

Pada kelas *Engine*, model berhasil mengklasifikasikan 9 dari 10 data secara benar, sedangkan 1 data salah diprediksi sebagai kelas Skid. Hal ini menunjukkan bahwa karakteristik suara mesin telah dikenali dengan cukup baik oleh model meskipun masih terdapat sedikit kemiripan dengan suara gesekan ban.

Secara keseluruhan, confusion matrix menunjukkan bahwa model mampu mengklasifikasikan sebagian besar data uji dengan baik. Kesalahan klasifikasi masih didominasi oleh kelas *Siren*, sedangkan kelas *Crash*, *Horn*, Skid, Normal, dan *Engine* menunjukkan tingkat keberhasilan klasifikasi yang relatif tinggi. Hasil ini menunjukkan bahwa model memiliki kemampuan yang cukup baik dalam membedakan enam kelas suara yang digunakan pada penelitian

4.5.4 Analisis Classification Report

Untuk memperoleh informasi yang lebih rinci mengenai performa model pada setiap kelas suara, hasil pengujian dianalisis menggunakan classification report. Metode evaluasi ini menyajikan beberapa metrik, yaitu precision, *recall*, F1-score, dan support. Precision menunjukkan ketepatan model dalam memberikan prediksi terhadap suatu kelas, *recall* menunjukkan kemampuan model dalam mengenali seluruh data

yang sebenarnya berasal dari kelas tersebut, sedangkan F1-score merupakan rata-rata harmonis antara precision dan *recall* yang digunakan untuk menggambarkan keseimbangan performa model.

Tabel 4.5 Performa Model

Kelas	Precision	Recall	Fi-Score	Support
<i>Crash</i>	1,00	0,90	0,95	10
<i>Horn</i>	0,90	0,90	0,90	10
Skid	0,69	0,90	0,78	10
Normal	0,90	0,90	0,90	10
<i>Siren</i>	1,00	0,50	0,67	10
<i>Engine</i>	0,69	0,90	0,78	10
Accuracy	-	-	0,83	60
Macro Average	0,86	0,83	0,83	60
Weighted Average	0,86	0,83	0,83	60

Berdasarkan Tabel 4.6, performa model menunjukkan hasil yang berbeda pada setiap kelas suara. Kelas *Crash* memperoleh performa terbaik dengan nilai precision sebesar 1,00, *recall* sebesar 0,90, dan F1-score sebesar 0,95. Hasil tersebut menunjukkan bahwa model mampu mengenali suara tabrakan dengan sangat baik dan menghasilkan tingkat kesalahan prediksi yang rendah.

Kelas *Horn* dan Normal juga menunjukkan performa yang baik dengan nilai precision, *recall*, dan F1-score masing-masing sebesar 0,90. Hal ini menunjukkan bahwa model mampu membedakan kedua jenis suara tersebut secara konsisten pada sebagian besar data uji.

Pada kelas Skid dan *Engine*, model memperoleh precision sebesar 0,69, *recall* sebesar 0,90, dan F1-score sebesar 0,78. Nilai *recall* yang tinggi menunjukkan bahwa sebagian besar data pada kedua kelas berhasil dikenali dengan benar. Namun, nilai precision yang lebih rendah mengindikasikan bahwa masih terdapat beberapa prediksi dari kelas lain yang salah diklasifikasikan sebagai Skid maupun *Engine*.

Sementara itu, kelas *Siren* memperoleh precision sebesar 1,00, namun *recall* sebesar 0,50 dan F1-score sebesar 0,67. Kondisi ini menunjukkan bahwa setiap prediksi yang diberikan model untuk kelas *Siren* cenderung benar, tetapi model masih belum mampu mengenali seluruh data Sirene yang tersedia sehingga sebagian data masih salah diklasifikasikan ke kelas lain.

Secara keseluruhan, model memperoleh accuracy sebesar 81,33%, dengan macro average precision sebesar 0,86, *macro average recall* sebesar 0,83, dan *macro average F1-score* sebesar 0,83. Nilai weighted average juga menunjukkan hasil yang sama, yaitu precision sebesar 0,86, *recall* sebesar 0,83, dan F1-score sebesar 0,83. Hasil tersebut menunjukkan bahwa model memiliki kemampuan klasifikasi yang cukup baik dalam membedakan keenam kelas suara yang digunakan pada penelitian ini. Hasil analisa tersebut belum melakukan proses fitting model.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan mengenai klasifikasi suara kecelakaan lalu lintas menggunakan metode *Mel Frequency Cepstral Coefficients* (MFCC) dan *Convolutional Neural Network* (CNN), maka dapat diambil beberapa kesimpulan sebagai berikut.

1. Penelitian ini berhasil membangun model klasifikasi suara kecelakaan lalu lintas menggunakan metode *Mel Frequency Cepstral Coefficients* (MFCC) sebagai proses ekstraksi fitur dan *Convolutional Neural Network* (CNN) sebagai algoritma klasifikasi. Model dibangun melalui beberapa tahapan, yaitu pengumpulan dataset, *preprocessing audio*, ekstraksi fitur MFCC, pelatihan model CNN, serta evaluasi menggunakan data uji.
2. Berdasarkan hasil evaluasi, model CNN memperoleh akurasi pengujian sebesar 81,33% dengan nilai *test loss* sebesar 0,5906. Hasil evaluasi menggunakan *precision*, *recall*, F1-score, dan confusion matrix menunjukkan bahwa model memiliki performa yang cukup baik dalam mengklasifikasikan enam kategori suara, yaitu *Crash*, *Horn*, *Skid*, *Normal*, *Siren*, dan *Engine*. Namun penelitian ini belum melakukan fitting.
3. Hasil penelitian menunjukkan bahwa kombinasi metode MFCC dan CNN mampu digunakan sebagai pendekatan yang efektif dalam proses klasifikasi suara kecelakaan lalu lintas. Hal ini menunjukkan bahwa fitur MFCC mampu merepresentasikan karakteristik sinyal *audio* dengan baik, sedangkan CNN mampu mempelajari pola-pola tersebut sehingga menghasilkan performa klasifikasi yang cukup baik pada enam kategori suara yang digunakan dalam penelitian.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, saran untuk pengembangan penelitian selanjutnya adalah sebagai berikut:

1. Menambah jumlah dataset pada setiap kelas agar model memperoleh variasi data yang lebih beragam sehingga kemampuan generalisasi model dapat meningkat.
2. Menambahkan variasi kondisi lingkungan, seperti kebisingan lalu lintas, hujan, maupun kondisi jalan yang berbeda agar model lebih adaptif terhadap kondisi nyata.
3. Mencoba metode ekstraksi fitur lain, seperti Mel Spectrogram, Chroma Features, atau Log-Mel Spectrogram, kemudian membandingkan performanya dengan metode MFCC.
4. Mengembangkan arsitektur CNN yang lebih kompleks atau membandingkannya dengan algoritma *deep learning* lainnya, seperti Long Short-Term Memory (LSTM), ResNet, atau EfficientNet, untuk memperoleh performa klasifikasi yang lebih optimal.
5. Mengimplementasikan model yang telah dibangun ke dalam sistem pemantauan lalu lintas atau perangkat *edge computing* sehingga dapat digunakan sebagai sistem klasifikasi suara secara *real-time*.

DAFTAR PUSTAKA

- Badan Pusat Statistik. (2025). *Jumlah kendaraan bermotor menurut jenis, 2020–2022*. Badan Pusat Statistik. Diakses pada 20 September 2025, dari https://www.bps.go.id/indicator/17/57/1/perkembangan-jumlah-kendaraanbermotor-menurut-jenis.html?utm_source=chatgpt.com
- Boylestad, R. L., & Nashelsky, L. (2020). *Electronic devices and circuit theory* (11th ed.). Pearson Education.
- Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.
- Deep learning*. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- Google Developers. (2024). *Classification: Accuracy, Precision, Recall, and Related Metrics*. Diakses pada 20 September 2025, dari <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>
- Indonesia. (2009). *Undang-Undang Republik Indonesia Nomor 22 Tahun 2009 tentang Lalu Lintas dan Angkutan Jalan*. Badan Pemeriksa Keuangan Republik Indonesia. Diakses pada 20 September 2025, dari <https://peraturan.bpk.go.id/Home/Details/39146/uu-no-22-tahun-2009>
- Islam, Z., & Abdel-Aty, M. (2022). *Deep Convolutional Neural Network for Roadway Incident Surveillance Using Audio Data*. arXiv. <https://arxiv.org/abs/2208.09547>.
- Kementerian Perhubungan Republik Indonesia. (2023). *Laporan lalu lintas dan kecelakaan*. Direktorat Jenderal Perhubungan Darat. Diakses pada 20 September 2025, dari <https://hubnet.kemenhub.go.id/dataset/content/dataset/29>
- Korps Lalu Lintas Kepolisian Negara Republik Indonesia. (2025). *Statistik Laka*. Diakses pada 20 September 2025, dari <https://korlantas.polri.go.id/?s=statistik+laka+lantas>
- Kumar, A., & Raj, B. (2020). *Deep CNN audio classifiers for real-time anomaly detection*. *Interspeech Proceedings*: https://www.isca-speech.org/archive/Interspeech_2020/abstracts/0849.html.

- Laurijssen, E., Guo, J., & Moonen, M. (2024). *Low-power MEMS microphone array processing for embedded audio detection*. *IEEE Sensors Journal*.
<https://ieeexplore.ieee.org/document/10345619>.
- Lee, J., & Kim, S. (2020). Detection of traffic accident sound using machine learning on Raspberry Pi. *International Journal of Distributed Sensor Networks*, 16(8), 1–10.
<https://doi.org/10.1177/1550147720943620>
- Li, T., Zhang, Y., & Qin, B. (2020). *Noise-Robust Audio Classification Using Data Augmentation and CNN*. *IEEE Access*:
<https://ieeexplore.ieee.org/document/9065603>. *Microcontrollers Lab INMP441 & ESP32 Tutorial*: <https://microcontrollerslab.com/esp32-inmp441-mems-microphone-tutorial/>.
- Lyons, R. G. (2020). *Understanding digital signal processing* (4th ed.). Pearson Education.
- Mulianingsih, M. (2024, 17 Desember). *KNKT catat 60% kecelakaan kendaraan darat karena pengemudi kelelahan*. DetikNews. diakses 20 September 2025
<https://news.detik.com/berita/d-7691062/knkt-catat-60-kecelakaan-kendaraan-darat-karena-pengemudi-kelelahan>
- Paranayapa, T., Ranasinghe, P., Ranmal, D., Meedeniya, D., & Perera, C. (2024). A comparative study of *preprocessing* and model compression techniques in deep learning for forest sound classification. *Sensors*, 24(4), 1149.
<https://doi.org/10.3390/s24041149>
- Sedra, A. S., & Smith, K. C. (2015). *Microelectronic circuits* (7th ed.). Oxford University Press.
<https://global.oup.com/academic/product/microelectronic-circuits-9780199339136>
- Singh, R., Kumar, S., & Verma, A. (2023). *Real-time road accident detection based on audio signal processing and deep learning*. *International Journal of Advanced Computer Science and Applications*, 14(6), 412–420.

Szeghalmy, S., & Fazekas, A. (2023). A comparative study of the use of stratified cross-validation and distribution-balanced stratified cross-validation in imbalanced learning. *Sensors*, 23(4), 2333. <https://doi.org/10.3390/s23042333>

TensorFlow. (2023). *Keras: The high-level API for TensorFlow*. <https://www.tensorflow.org/guide/keras>

Virtanen, T., Plumbley, M. D., & Ellis, D. P. W. (2020). *Computational analysis of sound scenes and events*. Springer. <https://doi.org/10.1007/978-3-030-11259-2>

Zhang, Y., Liu, H., & Wang, J. (2021). *Environmental sound classification for traffic accident detection using convolutional neural networks*. *IEEE Access*, 9, 120345–120356. <https://doi.org/10.1109/ACCESS.2021.3109123>

Zhao, X., Wang, L., Zhang, Y., Han, X., Deveci, M., & Parmar, M. (2024). *A review of convolutional neural networks in computer vision*. **Artificial Intelligence Review**, 57, 99. <https://doi.org/10.1007/s10462-024-10721-6>

IBI KESATUAN BOGOR

LAMPIRAN

Lampiran 1 Pelatihan Model CNN

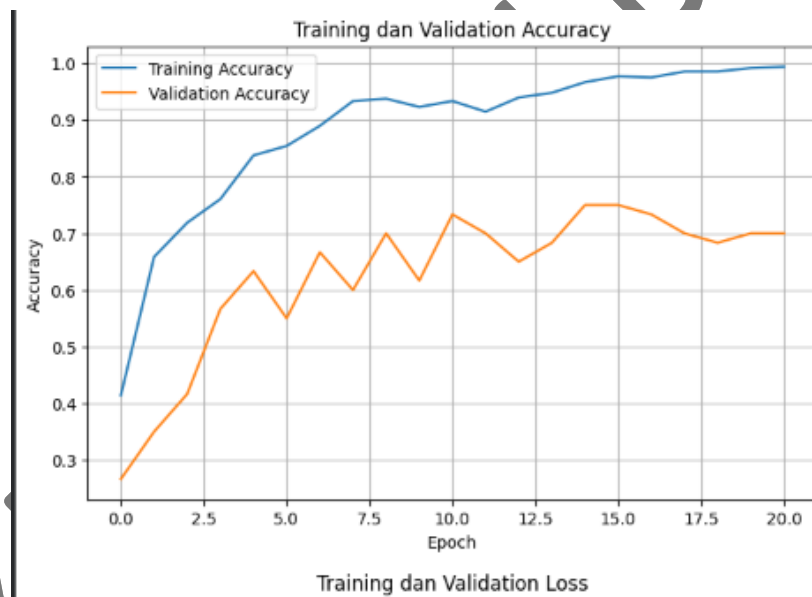
```

Epoch 14: ReduceLRonPlateau reducing learning rate to 0.0005000000237487257.
30/30 ----- 1s 48ms/step - accuracy: 0.9479 - loss: 0.1641 - val_accuracy: 0.6833 - val_loss: 1.8991 - learning_rate: 0.0010
Epoch 15/100 ----- 0s 44ms/step - accuracy: 0.9588 - loss: 0.1391
Epoch 15: val_accuracy improved from 0.73333 to 0.75000, saving model to /content/drive/MyDrive/Skripsi_Klasifikasi_audio/best_model.keras
Epoch 15: finished saving model to /content/drive/MyDrive/Skripsi_Klasifikasi_audio/best_model.keras
30/30 ----- 2s 58ms/step - accuracy: 0.9667 - loss: 0.1121 - val_accuracy: 0.7500 - val_loss: 1.8926 - learning_rate: 5.0000e-04
Epoch 16/100 ----- 0s 46ms/step - accuracy: 0.9734 - loss: 0.1076
Epoch 16: val_accuracy did not improve from 0.75000
30/30 ----- 1s 47ms/step - accuracy: 0.9771 - loss: 0.0932 - val_accuracy: 0.7500 - val_loss: 1.1280 - learning_rate: 5.0000e-04
Epoch 17/100 ----- 0s 44ms/step - accuracy: 0.9790 - loss: 0.0659
Epoch 17: val_accuracy did not improve from 0.75000
30/30 ----- 1s 47ms/step - accuracy: 0.9750 - loss: 0.0729 - val_accuracy: 0.7333 - val_loss: 1.0700 - learning_rate: 5.0000e-04
Epoch 18/100 ----- 0s 44ms/step - accuracy: 0.9785 - loss: 0.0691
Epoch 18: val_accuracy did not improve from 0.75000
30/30 ----- 1s 47ms/step - accuracy: 0.9854 - loss: 0.0630 - val_accuracy: 0.7000 - val_loss: 1.1252 - learning_rate: 5.0000e-04
Epoch 19/100 ----- 0s 51ms/step - accuracy: 0.9817 - loss: 0.0474
Epoch 19: val_accuracy did not improve from 0.75000

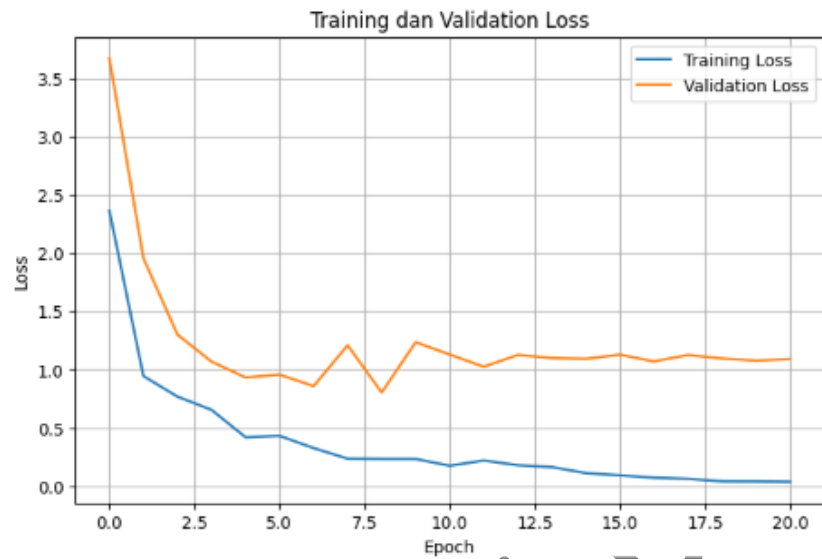
Epoch 19: ReduceLRonPlateau reducing learning rate to 0.0002500000118743628.
30/30 ----- 2s 58ms/step - accuracy: 0.9854 - loss: 0.0416 - val_accuracy: 0.6833 - val_loss: 1.0953 - learning_rate: 5.0000e-04
Epoch 20/100 ----- 0s 76ms/step - accuracy: 0.9945 - loss: 0.0342
Epoch 20: val_accuracy did not improve from 0.75000
30/30 ----- 2s 81ms/step - accuracy: 0.9917 - loss: 0.0407 - val_accuracy: 0.7000 - val_loss: 1.0769 - learning_rate: 2.5000e-04
Epoch 21/100 ----- 0s 74ms/step - accuracy: 0.9990 - loss: 0.0244
Epoch 21: val_accuracy did not improve from 0.75000
30/30 ----- 2s 78ms/step - accuracy: 0.9937 - loss: 0.0374 - val_accuracy: 0.7000 - val_loss: 1.0905 - learning_rate: 2.5000e-04
Epoch 21: early stopping
Restoring model weights from the end of the best epoch: 9.
~~~~~
waktu Training : 43.02 detik

```

Lampiran 2 Training dan Validation Accuracy



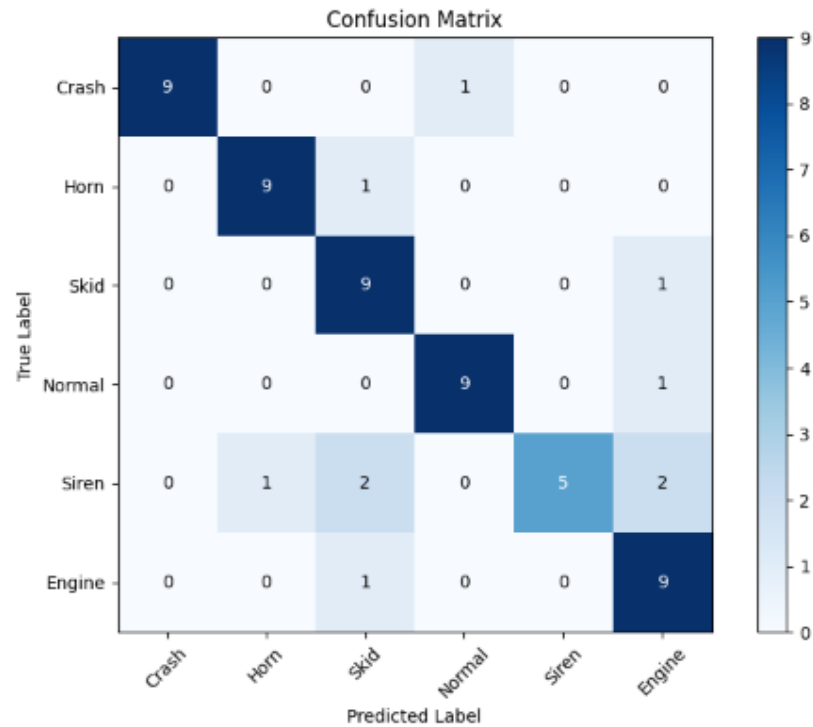
Lampiran 3 Training dan Validation Loss



Lampiran 4 Classification Report

***	precision	recall	f1-score	support
Crash	1.00	0.90	0.95	10
Horn	0.90	0.90	0.90	10
Skid	0.69	0.90	0.78	10
Normal	0.90	0.90	0.90	10
Siren	1.00	0.50	0.67	10
Engine	0.69	0.90	0.78	10
accuracy			0.83	60
macro avg	0.86	0.83	0.83	60
weighted avg	0.86	0.83	0.83	60

Lampiran 5 *Convusion Matrix*



Lampiran 6 *Splitting Data*

```
import os
import shutil
from sklearn.model_selection import train_test_split

INPUT =
"/content/drive/MyDrive/Skripsi_Klasifikasi_audio/dataset_preprocessed"
```

```

OUTPUT =
"/content/drive/MyDrive/Skripsi_Klasifikasi_audio/dataset_split"

classes = [
    "crash",
    "horn",
    "skid",
    "normal",
    "siren",
    "engine"
]

# Buat folder output
for split in ["train", "val", "test"]:
    for cls in classes:
        os.makedirs(os.path.join(OUTPUT, split, cls),
exist_ok=True)

for cls in classes:

    folder = os.path.join(INPUT, cls)

    files = sorted(os.listdir(folder))

    # 80% train
    train_files, temp_files = train_test_split(
        files,
        test_size=0.20,
        random_state=42,
        shuffle=True
    )

    # 10% val, 10% test
    val_files, test_files = train_test_split(
        temp_files,
        test_size=0.50,

```

```

        random_state=42,
        shuffle=True
    )

    # Copy file
    for f in train_files:
        shutil.copy(
            os.path.join(folder, f),
            os.path.join(OUTPUT, "train", cls, f)
        )

    for f in val_files:
        shutil.copy(
            os.path.join(folder, f),
            os.path.join(OUTPUT, "val", cls, f)
        )

    for f in test_files:
        shutil.copy(
            os.path.join(folder, f),
            os.path.join(OUTPUT, "test", cls, f)
        )

    print(f"{cls} ->
Train:{len(train_files)} Val:{len(val_files)} Test:{len(test_files)}")

```

Lampiran 7 Ekstrak Dataset

```

def extract_dataset(split):

    X = []
    y = []

```

```

for cls in classes:

    folder = os.path.join(DATASET_PATH, split, cls)

    files = sorted(os.listdir(folder))

    print(f"{split} - {cls} ({len(files)})")

    for file in tqdm(files):

        path = os.path.join(folder, file)

        audio, sr = librosa.load(
            path,
            sr=SAMPLE_RATE,
            mono=True
        )

        mfcc = librosa.feature.mfcc(
            y=audio,
            sr=sr,
            n_mfcc=N_MFCC,
            n_fft=N_FFT,
            hop_length=HOP_LENGTH
        )

        mfcc = mfcc.T

        X.append(mfcc)

        y.append(label_map[cls])

    return np.array(X), np.array(y)

```

Lampiran 8 Bangun Arsitektur CNN

```
def build_model():  
    model = Sequential()  
  
    model.add(  
        Conv2D(  
            32,  
            (3,3),  
            activation="relu",  
            input_shape=(94,20,1)  
        )  
    )  
  
    model.add(BatchNormalization())  
  
    model.add(MaxPooling2D((2,2)))  
  
    model.add(  
        Conv2D(  
            64,  
            (3,3),  
            activation="relu"  
        )  
    )  
  
    model.add(BatchNormalization())  
  
    model.add(MaxPooling2D((2,2)))  
  
    model.add(Flatten())  
  
    model.add(Dense(128, activation="relu"))  
  
    model.add(Dropout(0.3))
```

```

model.add(Dense(6, activation="softmax"))

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

return model

model = build_model()

```

Lampiran 9 *Training Model CNN*

```

import time

start_train = time.time()

history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=16,
    callbacks=[
        early_stop,
        checkpoint,
        reduce_lr
    ],
    verbose=1
)

```

```

end_train = time.time()

print("="*50)
print(f"Waktu Training : {end_train - start_train:.2f} detik")
print("="*50)

```

Lampiran 10 Evaluasi Model

```

import time

start_test = time.time()

test_loss, test_accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

end_test = time.time()

print("="*50)
print(f"Test Accuracy : {test_accuracy:.4f}")
print(f"Test Loss      : {test_loss:.4f}")
print(f"Waktu Testing : {end_test - start_test:.4f} detik")
print("="*50)

```

Lampiran 11 *Classification Report*

```

from sklearn.metrics import classification_report

```

```

class_names = [

    "Crash",

    "Horn",

    "Skid",

    "Normal",

    "Siren",

    "Engine"

]

print(classification_report(

    y_true,

    y_pred_classes,

    target_names=class_names

))

```

Lampiran 12 *Convusion Matrix*

```

from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

cm = confusion_matrix(y_true, y_pred_classes)

plt.figure(figsize=(8,6))

```

```

plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
plt.colorbar()

tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names, rotation=45)
plt.yticks(tick_marks, class_names)

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(
            j,
            i,
            str(cm[i, j]),
            ha="center",
            va="center",
            color="white" if cm[i, j] > cm.max()/2 else "black"
        )

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.show()

```